Physical Agents & World Models — Zero to Hero

An M-Files field guide to machines that see, remember, and help act in the physical world

Volume 2 · 29 September 2026 · written for the VMAN Vision team · volume history

Why this book exists

In 2026, "physical AI" stopped being a research topic and became a line item in factory budgets. Robots are being committed to production lines, and enterprise software vendors are racing to become the layer those robots trust before they act.

This book answers three questions, in plain language:

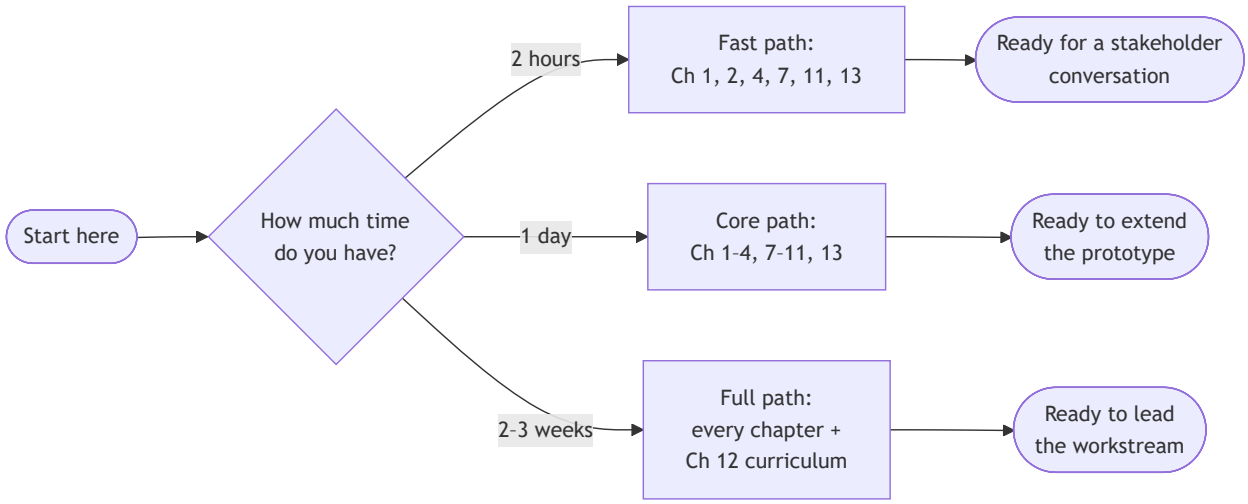
1. **What is all this?** Physical agents, world models, vision-language models, spatial grounding — what each one actually is, without the hype.
2. **Where does M-Files stand?** An honest map of our strengths and weaknesses, built from our own strategy pages, our own repositories, and dated external sources.
3. **What do we need to learn to close the gap?** A curriculum that covers exactly the tech areas the VMAN Vision goal needs — no more, no less.

It is written for an engineer who is smart, busy, and new to this area. Every chapter starts with an everyday scenario, builds the intuition, and only then gives the formal definition. Almost every section has a diagram.

> **The one sentence to remember:** M-Files will not win by building robots or out-modelling Google and Microsoft. It can win by being the **governed memory** a physical agent consults before it acts — and the book shows exactly which pieces of that memory exist today and which are still missing.

How to use this book

The whole book takes about **6–8 hours** to read. You don't need to read it in order.



- **Fast path (2 hours)** — enough to hold a confident conversation with a product owner about what we're proposing and why.
- **Core path (1 day)** — enough to understand, run, and extend the prototype in this repository.
- **Full path (2–3 weeks)** — the complete zero-to-hero curriculum, including hands-on capstones.

Each chapter ends with a box called **"Where M-Files stands"** that connects the concept back to our actual situation. If you only read those boxes, you'll still learn the map.

Table of contents

#	Chapter	What you'll understand after reading it
1	The map: where M-Files stands	Our strengths, our gaps, and why this book is shaped the way it is
2	What is a physical agent?	The sense → think → act loop, and why a chatbot isn't one
3	How machines see	Pixels → features → meaning; OCR vs. vision understanding
4	Pointing, not describing	Spatial grounding — the capability our prototype demonstrates
5	World models: imagining before acting	Encoders, latent state, dynamics — and what a world model is *not*
6	From seeing to doing: VLAs	How robots learn to act, and why M-Files doesn't need to train them
7	Knowledge graphs: our secret weapon	Why a governed graph is the memory physical agents need
8	Retrieval that can see	RAG → multimodal RAG → agentic RAG, and source attribution
9	Agents, tools and MCP	Giving the model hands: tool calling, agent loops, MCP
10	Trust: evaluation, safety and governance	How to know it works, and how to keep it safe
11	Putting it together: VMAN Vision	The architecture, the competition, and "start small, land big"

#	Chapter	What you'll understand after reading it
12	The hero's path: a curriculum	What to study, in what order, with hands-on capstones
13	The handover: your first week	Repository tour, running the demo, open threads, next conversations
A	Glossary	Every term in one place
B	Sources and verification status	Where every claim comes from, and how fresh it is
—	Volume history	What changed in each volume, with a frozen PDF of every past volume

A rendered PDF of the current volume lives next to these files: `physical-agents-zero-to-hero.pdf`. Rebuild it with `python book/tools/build_pdf.py` after editing any chapter. Every published volume is also kept, unchanged, in `volumes/`. See VOLUMES.md for how to cut a new one.

A note on honesty

This book separates three kinds of statements, and so should you when you talk about this work:

Label	Meaning	Example
Demonstrated	We ran it in this repository and kept the evidence	Gemini Robotics-ER 2 returned point-and-label output on engineering drawings
Documented	A vendor or an internal page says it, but we haven't tested it	Gemini ER 2 can track objects across video frames
Hypothesis	We think it's true and propose to test it	Visual evidence improves metadata extraction

Mixing these up is the fastest way to lose a technical reviewer's trust. Keeping them apart is the fastest way to earn it.

Chapter 1 — The map: where M-Files stands

Before the journey, the map

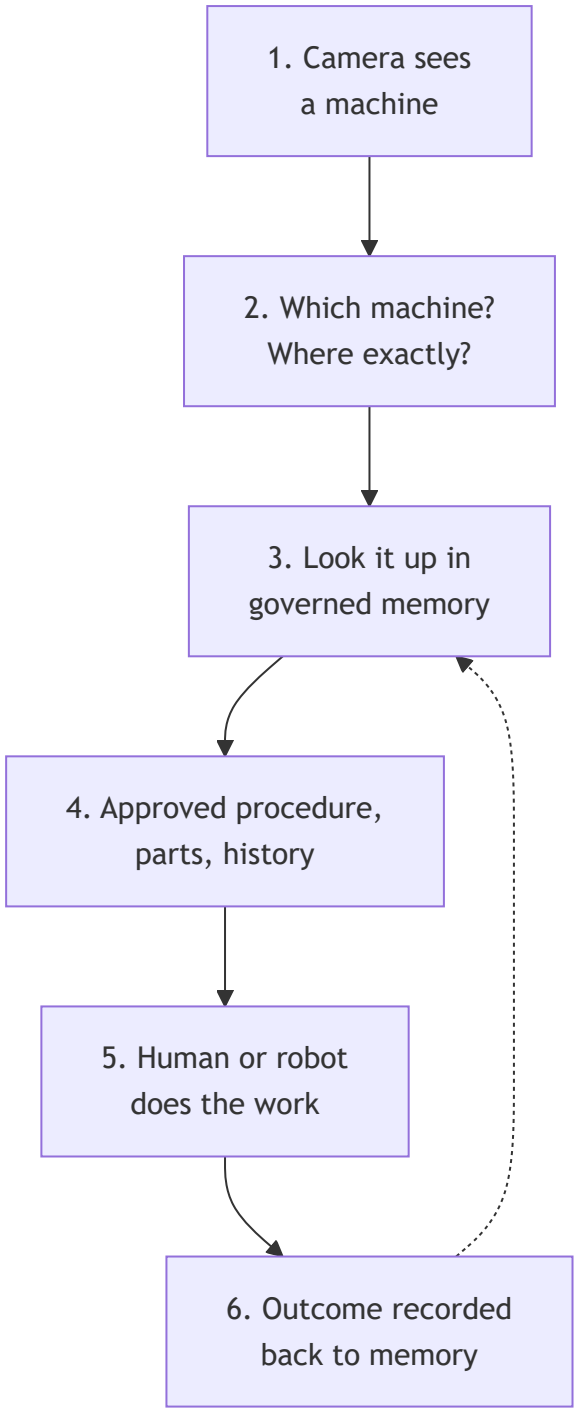
Imagine you're planning a hike up a mountain you've never climbed. Before you buy boots, you want two things: a map of the mountain, and an honest look at your own fitness. Are your legs strong? Is your lung capacity weak? The answers decide your training plan.

This chapter is that honest look — for M-Files, against the goal of VMAN Vision. Every later chapter teaches one of the areas this chapter identifies as a gap. That's why the book is shaped the way it is.

The goal, in one picture

The goal is simple to say and hard to build:

> A technician points a phone (or a robot points a camera) at a machine. The system recognises **which** machine it is, **where** on it the problem is, and returns the **approved, governed** procedure for fixing it — with permissions checked and the outcome recorded.



Every box in that picture is a technology area. Some M-Files already does well. Some it doesn't do at all yet.

How this map was built

The map comes from four kinds of evidence, not opinion:

1. **Our own strategy and roadmap pages** on Confluence — the Product Strategy 2025–26, the Manufacturing Blueprint, the 2027 forward-looking thoughts, and the CAD Integration Roadmap

2026–2029 [9]–[12].

2. **Our own repositories** — this one (`physical-agents-research`) and its sibling `complex-document-grounding-poc`, including what's been demonstrated versus merely proposed.
3. **Our own platform documentation** — the MCP Server beta page, the Aino architecture pages, and the AI-Metadata team's design documents as reviewed in `docs/differentiation-plan-vs-metadata-poc.md`.
4. **Dated external sources** — vendor documentation and industry reporting, listed in Appendix B.

Our strengths

Strength 1 — We already own a knowledge graph

M-Files doesn't store files in folders. It stores objects with metadata, and relationships between them — documents linked to projects, people, machines, and companies. The Product Strategy 2025–26 names this "knowledge-graph-like architecture" as how we win, and notes that "knowledge graphs are a favored format for AI" [9].

Why it matters: a physical agent needs memory. Not "search results" — *structured* memory: this pump belongs to that line, has this service history, and this is its approved maintenance procedure. That's a graph. We already have the shape. (Chapter 7.)

Strength 2 — Governance is in our DNA

Permissions, versioning, audit trails, approval workflows. The Manufacturing Blueprint puts it crisply: "your systems keep the transaction, M-Files keeps the governed context" [10].

Why it matters: before any enterprise lets an AI act in the physical world, someone will ask *"who approved that procedure, and who is allowed to see it?"* We are one of the few vendors for whom that's the core product, not an afterthought. (Chapter 10.)

Strength 3 — We don't compete with the systems robots plug into

The Blueprint states that M-Files does not compete with PLM, ERP, MES or EAM systems [10]. That makes us a natural *partner* for them rather than a threat.

Why it matters: the physical-AI race is being run by giants (Chapter 11). A partner-shaped product can plug into their stacks instead of fighting them.

Strength 4 — Microsoft alignment

Our strategy pairs the knowledge graph with AI and Microsoft [9]. Our AI stack (Aino) already runs on Azure services.

Why it matters: most of our customers already trust Microsoft with their data. An Azure-native path for vision is much easier to approve than an external dependency. (Chapters 8 and 10.)

Strength 5 — A working, honest prototype

This repository has a real, running vision-grounding demo, with side-by-side evidence from two providers on public engineering drawings, and documents that carefully separate what's proven from what isn't. (Chapter 4.)

Strength 6 — A route to prove things fast

The 2027 forward-looking thoughts describe Forward Deployed Engineering: proving ideas "against a real customer's budget and urgency … ahead of when the core roadmap would otherwise reach it" [11].

Why it matters: we don't have to wait for the core roadmap to test this. There is a sanctioned path. (Chapter 13.)

Our weaknesses (the gaps)

Gap 1 — We can't see yet (perception)

No production M-Files AI surface reads images today. The MCP Server beta explicitly lists "image reading is not available in Beta" as a tracked limitation. The CAD Integration Roadmap says non-CAD users see drawings as "opaque blobs" [12].

Taught in: Chapter 3 (how machines see), Chapter 4 (spatial grounding).

Gap 2 — Our search reads words, not pictures (retrieval)

Aino Search is built on text embeddings. The existing "Images" processing mode sends page images to the LLM only when files are small (≤10 MB) and silently falls back to text-only above that, per the AI-Metadata team's workflow documentation.

Taught in: Chapter 8 (retrieval that can see).

Gap 3 — We can't point back to *where* an answer came from (attribution)

The metadata-poc design documents reviewed on 2026-09-08 contain no concept of citing back to a region on a page. Only the sibling repository's Document Intelligence adapter currently preserves region-level evidence.

Taught in: Chapter 4 and Chapter 8.

Gap 4 — The asset memory is designed, not built

The Manufacturing Blueprint lists **Installed Base & Maintenance Knowledge** as *planned*: "equipment by serial number … approved maintenance procedures as governed steps an assistant renders verbatim" [10]. The 2027 thoughts confirm it's "validated design work, not yet built" [11]. The CAD roadmap's engineering-knowledge-graph milestone is dated H1 2028 [12].

Taught in: Chapter 7 (knowledge graphs) and Chapter 11 (architecture).

Gap 5 — We can't yet prove how well it works (evaluation)

Every comparison in this repository is qualitative. There is no labelled evaluation set, no scoring rubric, no measured accuracy. The repository's own documents say so.

Taught in: Chapter 10 (trust).

Gap 6 — Model access is awkward

The best spatial-grounding model we've tested (Gemini Robotics-ER 2) is a preview endpoint and isn't deployable inside Microsoft Foundry. The Foundry deployment we tested as a stand-in (`4Omni`, still routed under the legacy name `magma` in the demo) returned malformed output in two of six internal-drawing runs. Microsoft's own spatial model, Magma, is now listed in the Foundry model catalog. But it runs on dedicated-GPU "Managed Compute", which is still in preview, needs a quota request and depends on project/region availability. It's untested here, and its model card rules out production use (see `docs/magma-readiness.md`).

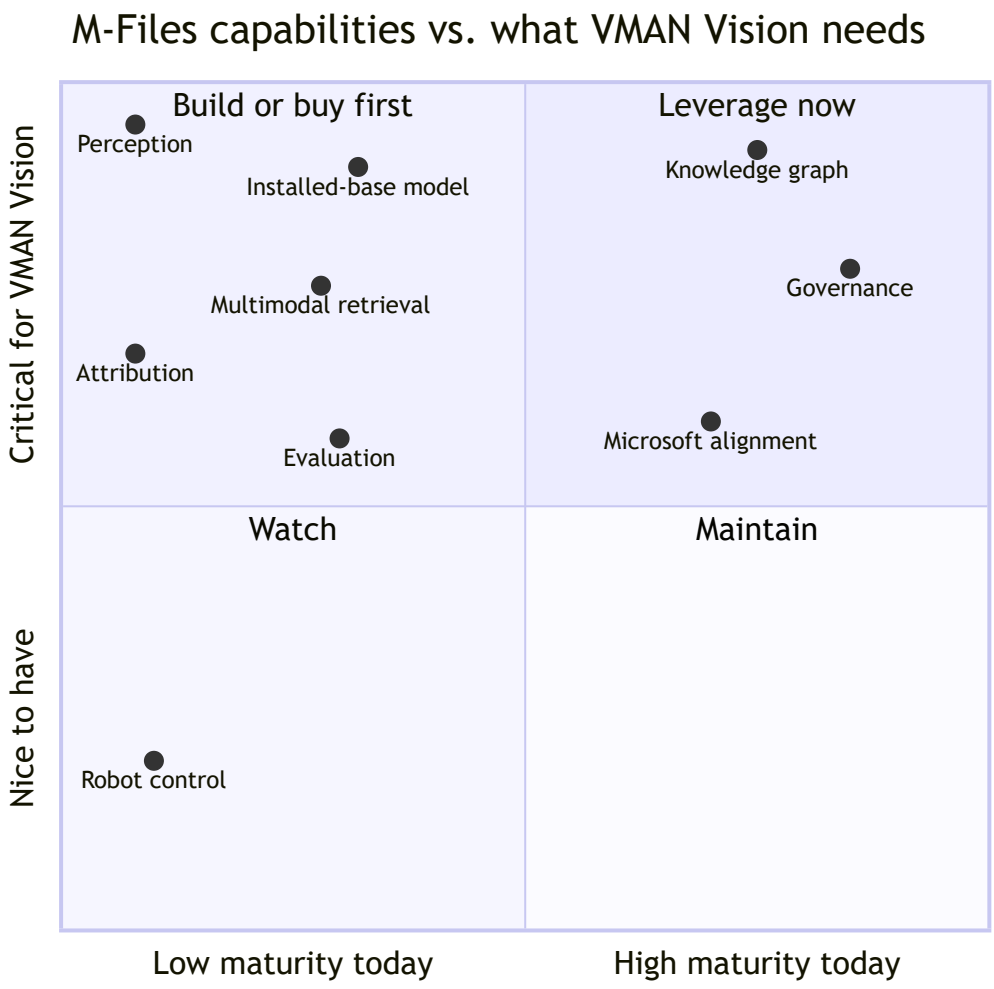
Taught in: Chapter 4 and Chapter 10.

Gap 7 — The clock

SAP, Palantir, Siemens and PTC are already shipping or piloting the "ground the robot in enterprise data" story (Chapter 11). Our engineering-knowledge-graph milestone is 2028.

Taught in: Chapter 11.

The map on one page



Read it like this: the **top-right** is what we should lean on in every conversation. The **top-left** is what this book mostly teaches — critical, and not yet mature. Robot control sits bottom-left on purpose: it's a real field, but it's not our job.

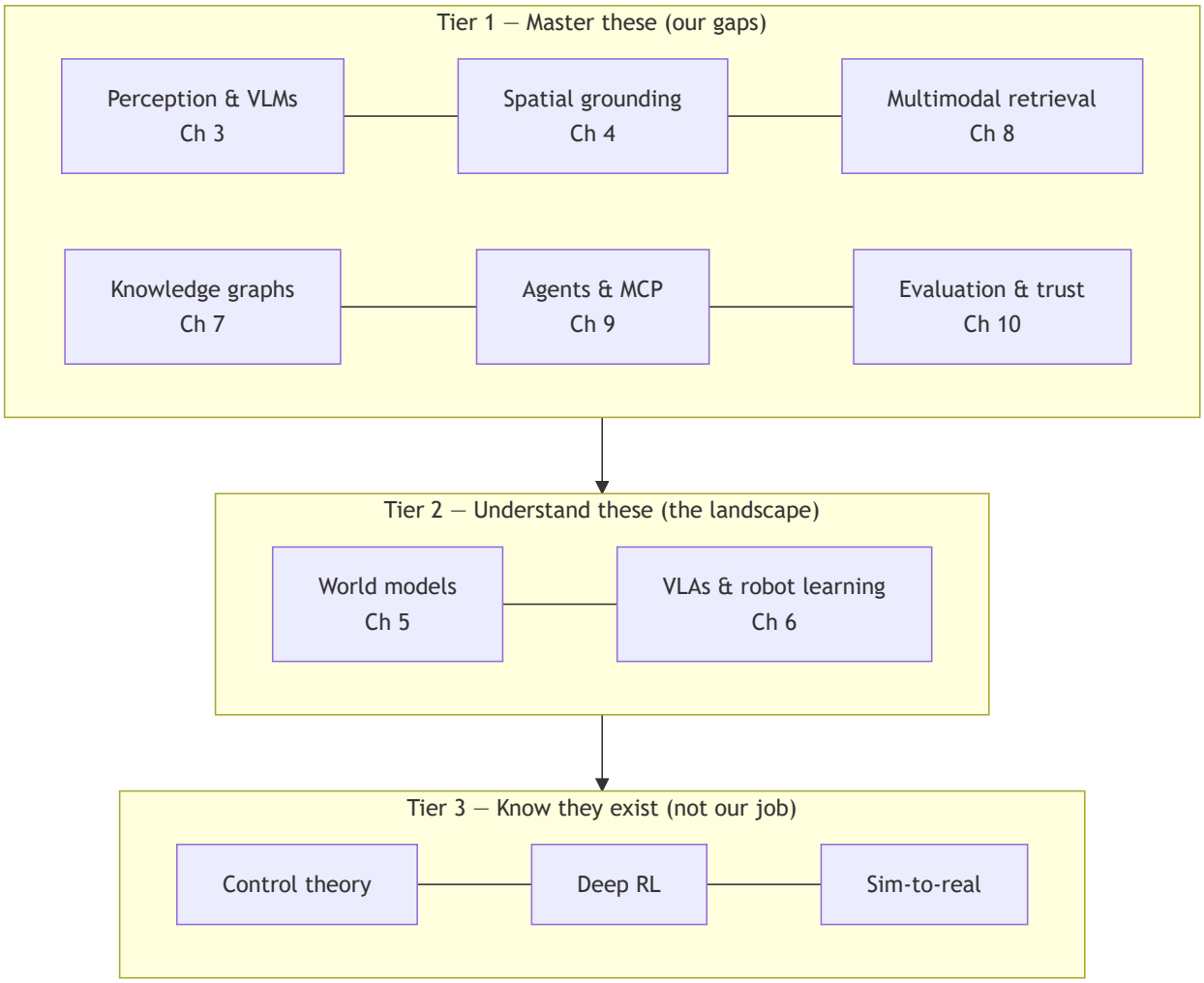
The capability matrix

Area	Today	Evidence	What it means for the goal	Chapter
Knowledge graph	**Strong**	Product Strategy 2025–26 [9]	The memory layer physical agents need	7
Governance	**Strong**	Manufacturing Blueprint [10]	The reason an enterprise would trust us	10
Partner positioning	**Strong**	Blueprint non-compete stance [10]	We plug into giants, not fight them	11
Microsoft alignment	**Strong**	Strategy [9], Aino on Azure	Easier approval path for vision	8, 10

Area	Today	Evidence	What it means for the goal	Chapter
Prototype evidence	**Partial**	This repo (qualitative only)	Proof the idea runs, not that it's accurate	4
Perception	**Missing**	MCP beta limitation; "opaque blobs" [12]	The first gap to close	3, 4
Multimodal retrieval	**Missing**	Aino text embeddings; 10 MB images cap	Search must see, not just read	8
Region attribution	**Missing**	metadata-poc review	"Show me where" builds trust	4, 8
Installed-base model	**Planned**	Blueprint [10], 2027 thoughts [11]	The graph nodes a photo must resolve to	7, 11
Evaluation	**Missing**	Repo docs: no labelled set	Without it, nothing is provable	10
Model access	**Awkward**	ER 2 preview, not in Foundry	Choose the path deliberately	4, 10
Robot control/training	**Out of scope**	By design	Understand it, don't build it	5, 6

What this means for the rest of the book

This book grew out of an earlier six-page personal guide, which was a general robotics curriculum: control theory, reinforcement learning, simulators. All useful — but the map above says M-Files' real gaps are elsewhere: **seeing, pointing, retrieving, remembering, and proving**. So this book flips the weight:



Where M-Files stands

Strong on memory and trust. Missing the eyes. We own the governed knowledge graph a physical agent would need to consult, and the governance an enterprise would demand. What we lack is the perception layer to connect a camera to that graph, the retrieval that can search pictures, and the evaluation discipline to prove any of it works. The rest of this book teaches exactly those pieces.

Chapter 2 — What is a physical agent?

Your first day on the factory floor

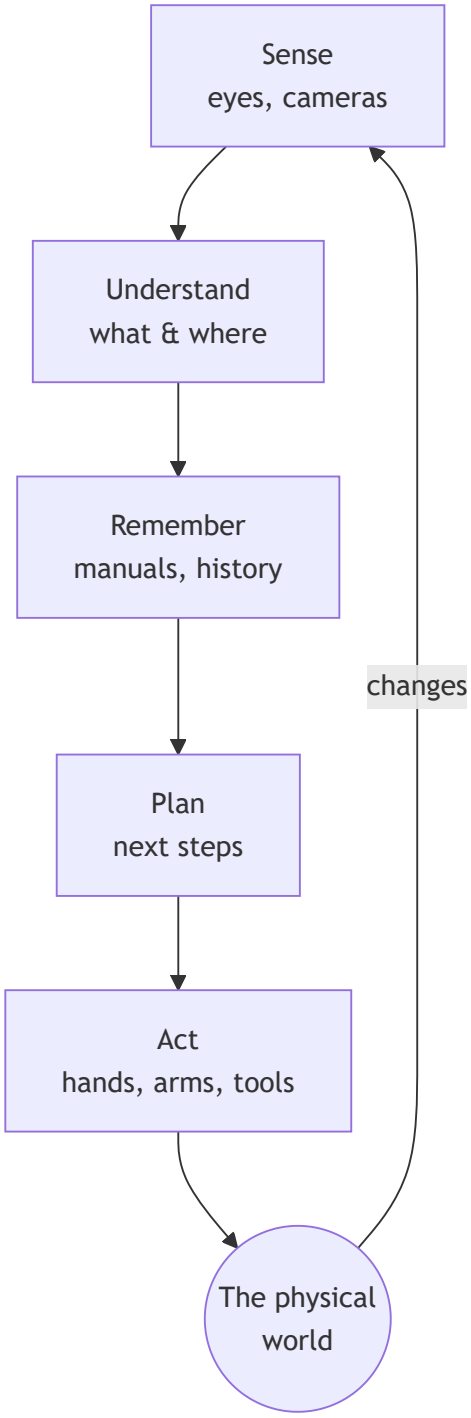
It's your first day as a maintenance technician. Your supervisor walks you to a humming machine and says:

"*The left pump is leaking. Fix it.*"

Think about what you do next, step by step:

1. **You look.** Your eyes take in the machine — pipes, valves, a puddle.
2. **You find.** "Left pump" — which one is that? You locate it, and the exact joint that's dripping.
3. **You remember.** You recall (or look up) what this pump model is, what the manual says, and what parts fit it.
4. **You plan.** Shut the valve, drain the line, replace the seal.
5. **You act.** Your hands do the work.
6. **You check.** You look again. Still dripping? Back to step 1.

You just ran a loop: **sense → understand → remember → plan → act → check.** Every physical agent — human, robot, or software helping a human — runs some version of it.



The loop never really ends, because the world keeps changing — including because of what you just did.

The formal definition

To formalise: **a physical agent is a system whose decisions must work through sensors, bodies or tools, in a real environment that changes — including in response to the agent's own actions.**

Three words in that definition carry the weight:

- **Sensors** — it can't just read text; it has to perceive the world (cameras, depth sensors, touch, sound).
- **Acts** — its output changes something physical, directly (a robot arm) or indirectly (a technician following its instructions).
- **Changes** — the world pushes back. Plans go stale. The agent has to check and adapt.

Chatbot vs. physical agent

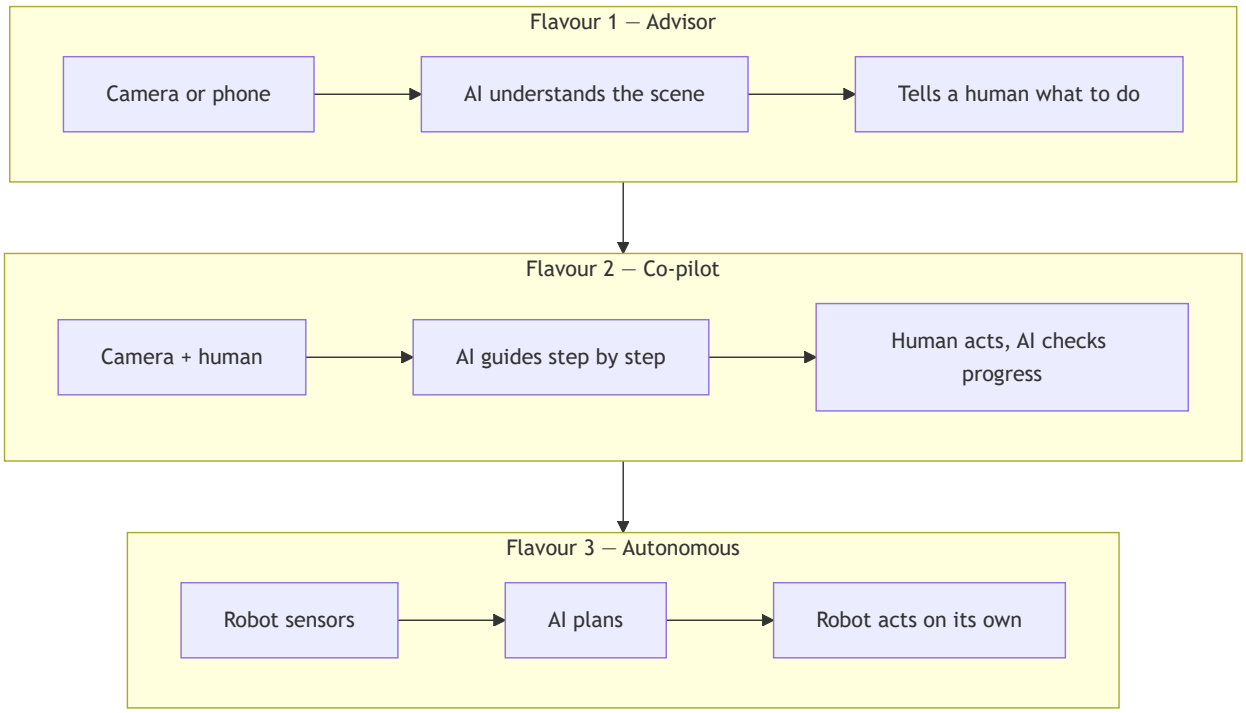
This is the most useful comparison in the whole book, so let's be precise.

	Chatbot (LLM assistant)	Physical agent
Input	Text (maybe an image you upload)	Continuous sensor streams
Output	Text	Actions, or instructions a human executes
World	Frozen — the conversation is the world	Live — it changes while you think
Mistakes cost	A wrong sentence	A broken part, a safety incident
"Done" means	The answer was written	The world is in the right state
Needs to know *where*?	Rarely	Always

That last row is the one to remember. A chatbot can say *"the valve near the tank."* A physical agent has to know *which pixel, which bolt, which millimetre.* This is why Chapter 4 — pointing — matters so much.

The three flavours of physical agent

Not every physical agent has a robot body. It helps to think of three flavours:



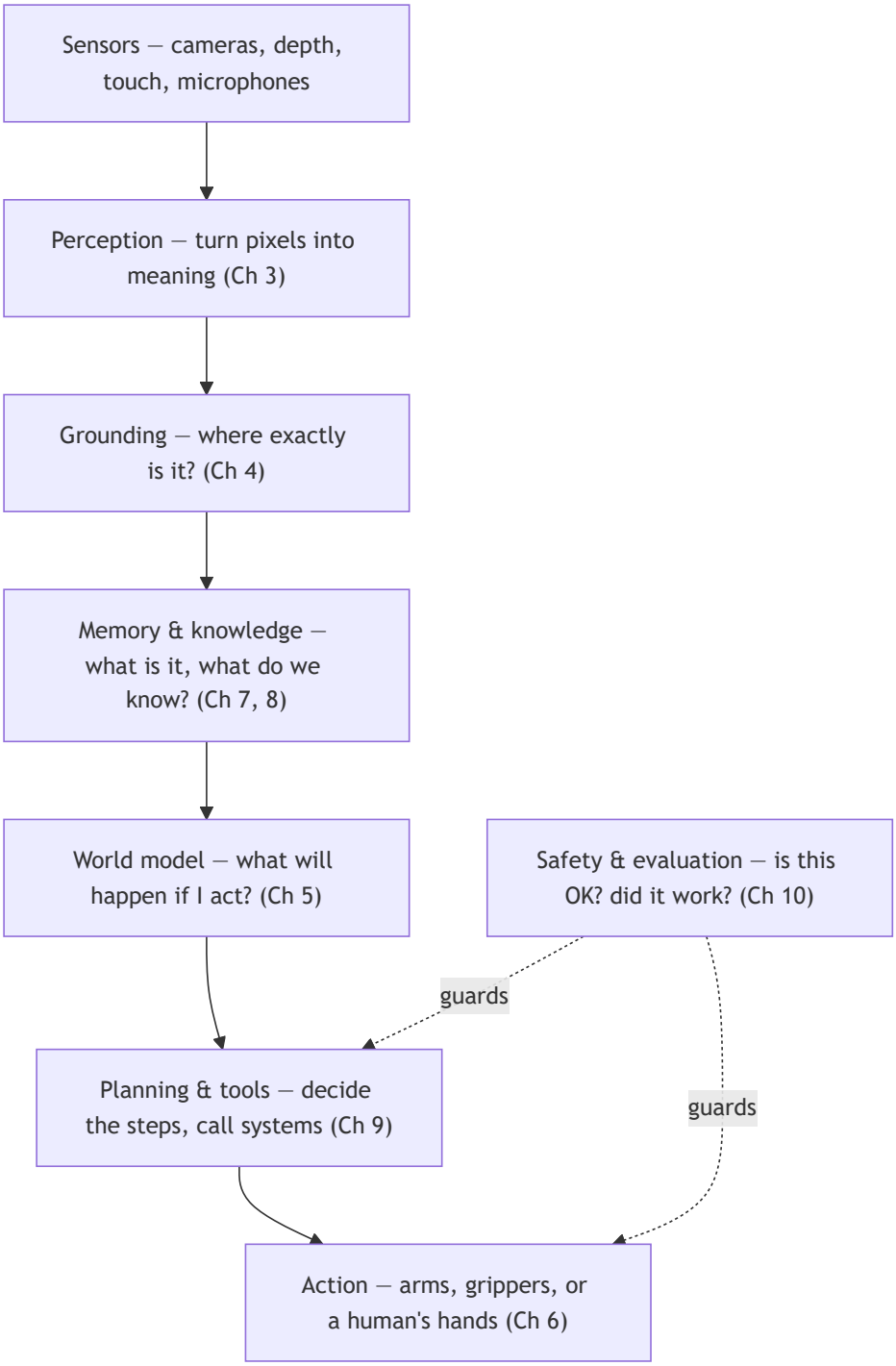
1. **Advisor** — a technician photographs a machine; the system identifies it and returns the right procedure. The human does everything physical.
2. **Co-pilot** — the system watches (e.g. through a tablet or headset) and guides, step by step, checking progress.
3. **Autonomous** — a robot perceives, plans, and acts itself.

Most of the industry noise is about flavour 3 — humanoids, robot arms. But flavours 1 and 2 are where most near-term enterprise value sits, and they need the same underlying capabilities: seeing, locating, remembering, and checking.

> **Important for M-Files:** the Manufacturing Blueprint says M-Files "records outcomes and recommends actions; it does not execute transactions" [10]. That maps cleanly onto the three flavours: M-Files is the memory and the advice in all three — it is never the hands.

What's inside a physical agent

Let's open the box. A modern physical agent is built from a stack of parts, and the rest of this book takes them one at a time:



Notice that safety isn't a layer at the end — it watches the planning and the action.

Three misunderstandings to avoid

The field moves fast, and people blur terms. Three common confusions:

1. **"An LLM is a world model."** Not automatically. An LLM predicts the next *word*. A world model predicts the next *state of the world* after an action. Language knowledge helps, but it isn't physics. (Chapter 5.)
2. **"If it can describe the image, it can act on it."** No. "*There's a valve on the right*" is a description. "*The valve is at pixel (870, 540), and turning it clockwise closes the outlet*" is actionable. Description is not grounding. (Chapter 4.)
3. **"A great benchmark score means it's safe."** Benchmarks measure a specific robot, dataset, and task. They don't prove safety under new lighting, new machines, or new failure modes. (Chapter 10.)

Why now?

Three things changed at roughly the same time:

- **Models learned to see well.** Vision-language models can now describe *and* locate things in images reliably enough to be useful.
- **Robot data exploded.** Large shared datasets and cheap video made it possible to train general-purpose robot policies, not one model per task.
- **Money arrived.** McKinsey sizes the 2040 general-purpose robotics market at about **$370B**, up from under $1B in 2025, with logistics and discrete manufacturing making up roughly 60% of it [5].

NVIDIA's Jim Fan puts the moment in one phrase: robotics is repeating the language-model playbook — scale data and compute, and capability emerges — and we're early in that curve [1].

> ### Where M-Files stands
>
> M-Files doesn't need to build robots to be part of this. In every flavour of physical agent — advisor, co-pilot, autonomous — the agent has to *remember*: what this machine is, what's been done to it, what the approved procedure is. That memory is M-Files' home turf. What we're missing is the *sensing* half: the ability to go from a camera image to the right object in our graph. That's the next two chapters.

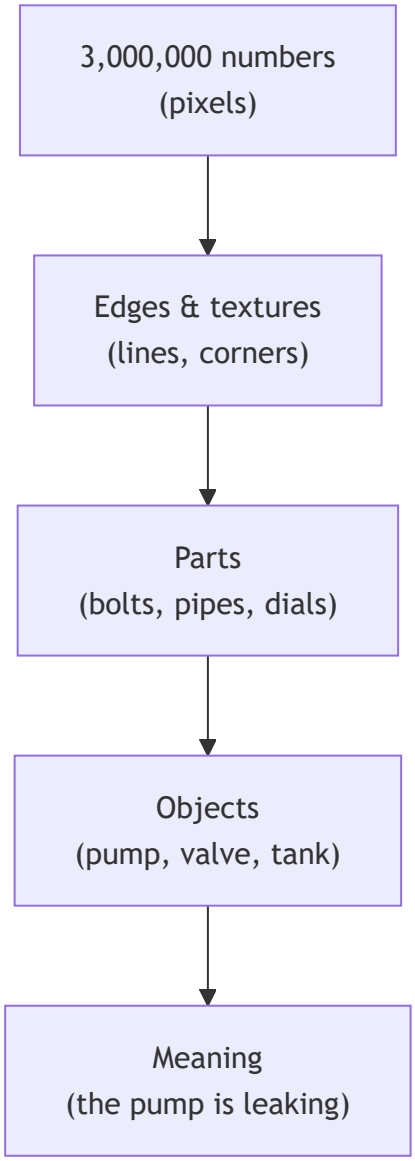
Chapter 3 — How machines see

A photo is just a spreadsheet of numbers

Open any photo on your laptop and zoom in until you see individual squares. Each square — a pixel — is three numbers: how much red, how much green, how much blue. A 1000×1000 photo is three million numbers.

That's all a computer "sees". There's no pump in there, no valve, no leak. Just a very large spreadsheet of brightness values.

So the whole problem of computer vision is this: **how do you get from three million numbers to "the left pump is leaking"?**



Each step compresses the numbers into something more meaningful. Modern vision models learn these steps from data rather than having them hand-coded.

Step 1 — Finding edges and patterns (CNNs)

Imagine sliding a small magnifying glass over the photo, one patch at a time, asking a single question: *"is there a vertical edge here?"* Then another glass asks *"is there a curve?"* Another: *"a circle?"*

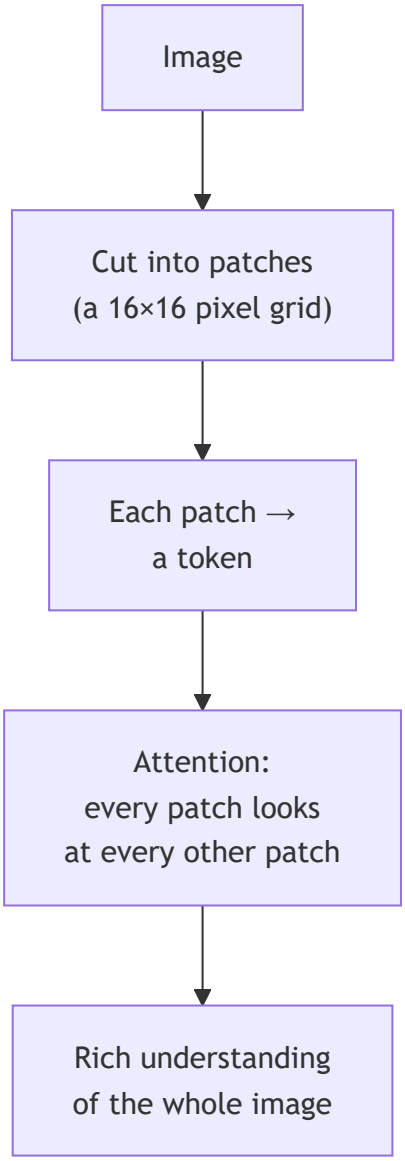
That's a **convolutional neural network (CNN)**. Small filters slide across the image; early layers learn edges, middle layers learn shapes, late layers learn whole objects. CNNs (made famous by AlexNet in 2012 and ResNet in 2015) are what made computer vision practical at scale.

To formalise: **a CNN is a neural network that applies small learned filters across an image to build up a hierarchy of features, from edges to objects.**

Step 2 — Looking at everything at once (Vision Transformers)

CNNs look locally — patch by patch. But understanding a drawing often needs *long-range* connections: the label in the corner explains the symbol in the middle.

The **Vision Transformer (ViT)** takes a different approach. It cuts the image into a grid of patches (say 16×16 pixels each), turns each patch into a list of numbers called a *token*, and lets every patch "pay attention" to every other patch.

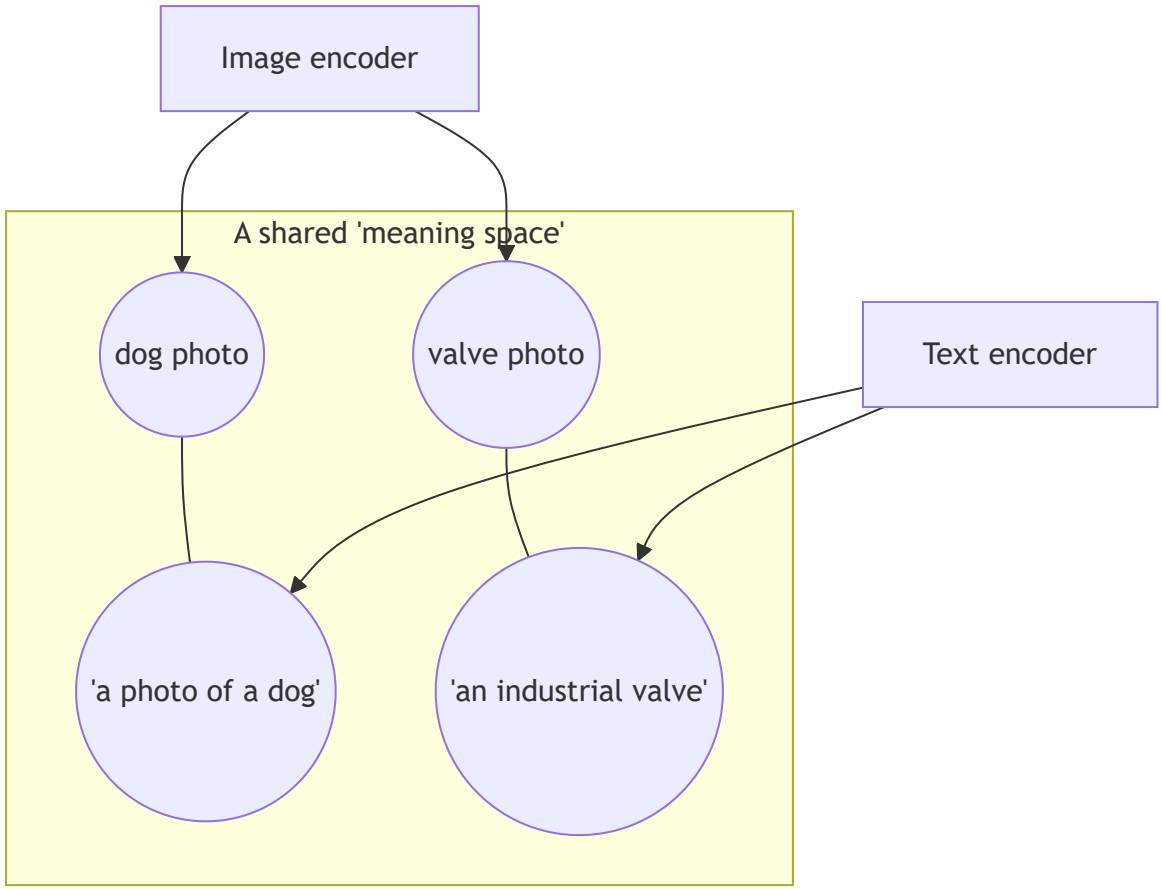


This is the same attention mechanism that powers language models — which turned out to matter enormously, because it let vision and language be combined in one model.

Step 3 — Connecting pictures to words (CLIP)

Here's a clever trick. Take 400 million images from the internet, each with its caption. Train two encoders — one for images, one for text — so that a photo of a dog and the sentence "*a photo of a dog*" end up as *nearby* points in the same mathematical space.

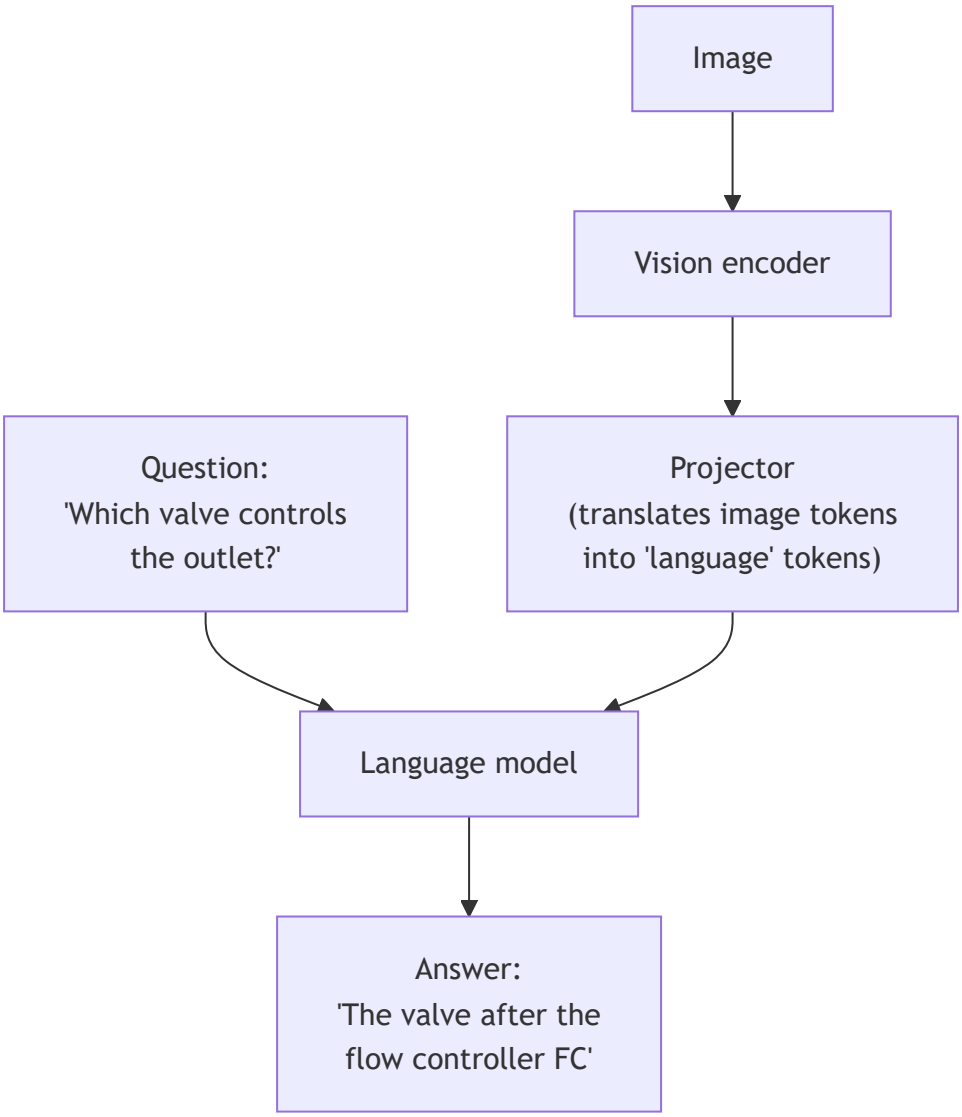
That's **CLIP** (OpenAI, 2021). After training, you can compare any image to any sentence, even ones it never saw.



This idea — **putting images and text in the same space** — is the foundation of modern multimodal AI, and of multimodal search (Chapter 8).

Step 4 — Models that see *and* talk (VLMs)

Now combine it all. Take a vision encoder (a ViT), feed its tokens into a large language model, and train the pair together. You get a **vision-language model (VLM)**: you can show it an image and ask questions in plain language.



To formalise: **a VLM is a model that jointly processes images and text, so it can answer questions about images, describe them, and — in the better ones — locate things in them.**

GPT-4o, Gemini, and Claude are all VLMs. Gemini Robotics-ER 2 is a VLM specialised for spatial and physical reasoning (Chapter 4).

OCR vs. vision understanding

This distinction is at the heart of M-Files' current gap, so it deserves its own section.

OCR (optical character recognition) answers one question: *what are the characters on this page?* It's decades old, extremely reliable for printed text, and still the right tool for reading text.

Vision understanding answers different questions: *what does this diagram mean? Which symbol is the pump? How is this valve connected to that tank?*

	OCR	Layout analysis	Vision understanding (VLM)
Question it answers	What text is here?	Where are the text blocks, tables, figures?	What does this mean, and where is it?
Output	Characters and words	Regions, tables, reading order	Descriptions, answers, points, boxes
Engineering drawing	Reads the labels	Finds the title block	Understands the symbols and connections
Chart	Reads axis numbers	Finds "a figure"	Reads the trend
Reliability	Very high	High	Good but can hallucinate
Example	Classic OCR engines	Azure Document Intelligence	Gemini, GPT-4o

Try running classic OCR on a P&ID (piping and instrumentation diagram). You'll get a scatter of tags — "FC", "PI", "LP Steam" — with no idea how they connect. That's why our CAD Integration Roadmap describes drawings as "opaque blobs" to non-CAD users [12]. The text is there. The *meaning* isn't.

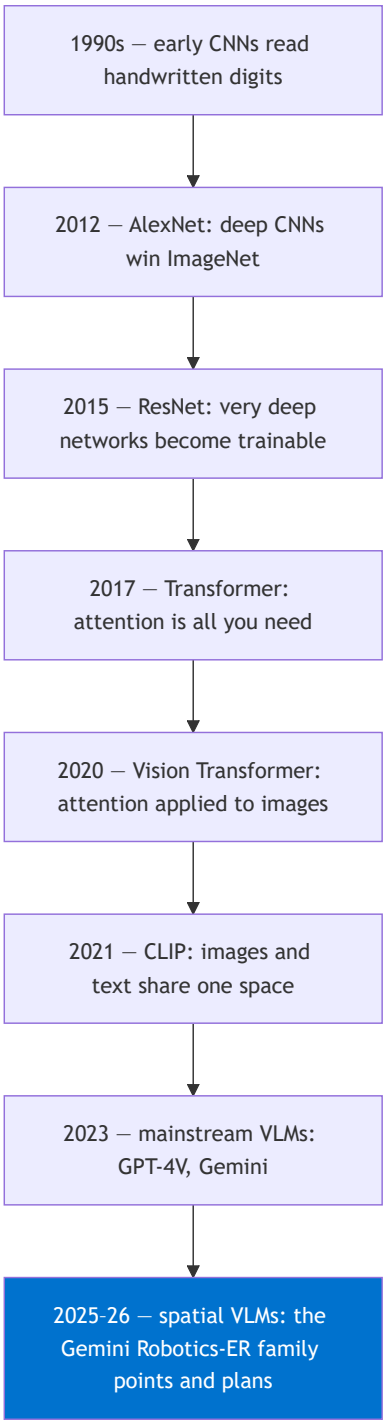
Where vision models still struggle

VLMs are impressive, but a good engineer knows the failure modes:

- **Tiny details.** Models usually downscale images. A 6-point font dimension on an A0 drawing can vanish. (Fix: crop and re-inspect — Chapter 4.)
- **Counting.** "How many bolts?" is surprisingly hard. Models guess.
- **Hallucination.** A model may confidently describe a valve that isn't there. Google's own documentation warns that spatial outputs can be wrong, especially on unusual inputs [17].
- **Domain symbols.** Standard P&ID symbols are learnable; a company's in-house symbol conventions may not be.
- **Consistency.** The same image and prompt can give slightly different answers on different runs.

None of these are reasons not to use VLMs. They are reasons to *always check the output visually* and to build an evaluation set before making claims (Chapter 10).

The timeline in one picture



Where M-Files stands

Missing, but buyable. M-Files doesn't need to train a vision model — world-class ones are available as APIs from Google and Microsoft. What's missing is the *wiring*: no production M-Files surface sends images to a vision model and does something useful with the answer. The MCP Server beta explicitly can't read images. The good news: this is the most solvable gap on the map, because the hard part (the model) already exists. The next chapter covers the specific vision capability that matters most for us.

Chapter 4 — Pointing, not describing

"The valve on the right"

You're on a phone call with a colleague who's standing in front of a machine. You're looking at the drawing. You say:

"*Close the valve on the right.*"

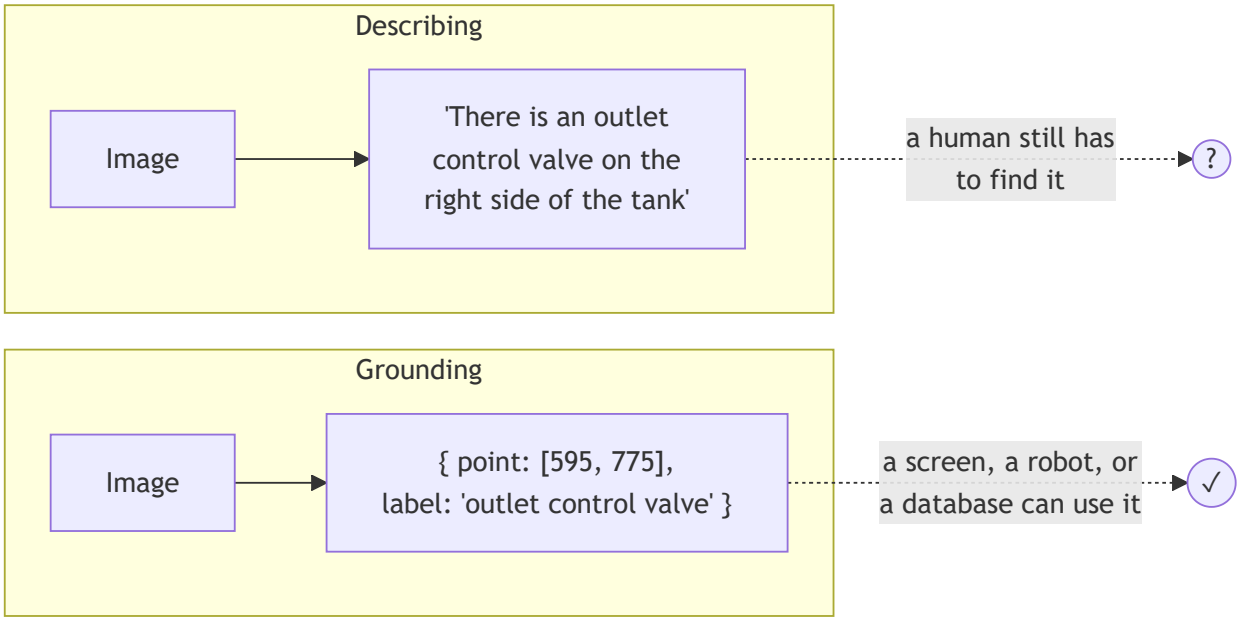
Silence. Then: "*There are four valves on the right.*"

You try again: "*The one just after the flow controller, before the outlet.*" Better. But what you really want to do is reach through the phone and **put your finger on it.**

That's the difference this chapter is about. Describing a thing is not the same as locating it. Language is great at the first. Physical work needs the second.

What spatial grounding is

To formalise: **spatial grounding is the ability of a model to connect words to locations — to answer not just "what is in this image?" but "where exactly is it?"**, as coordinates you can draw, measure, or send to a machine.



A description needs a human to interpret it. A coordinate can be drawn on a screen, matched to a region of a drawing, or passed to a robot arm.

How coordinates are expressed: the 0–1000 grid

Images come in every size. A phone photo might be 4000×3000 pixels; a scanned A0 drawing might be 14000×10000. If a model answered in raw pixels, the answer would depend on the size of the file you sent.

So spatial models use a **normalised grid**. Gemini Robotics-ER 2 returns points as **[y, x]**, each on a scale from **0 to 1000**, regardless of image size:

```
(0, 0) ──────────────────────────── x → (0, 1000)
  │
  │           • [400, 575]  "tank"
  │
  y                 • [595, 775] "outlet valve"
  ↓
(1000, 0)                          (1000, 1000)
```

Two details trip everyone up the first time:

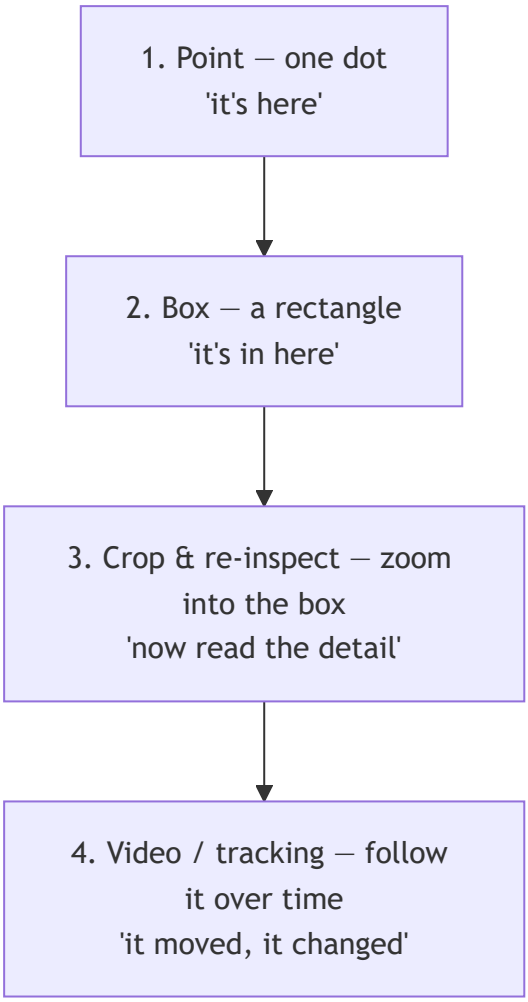
- **y comes first.** It's `[y, x]`, not `[x, y]`. Mix them up and every point lands in the wrong place.
- **It's a fraction, not a pixel.** 500 means "halfway", whether the image is 800 or 8000 pixels wide.

Converting back to pixels is one line of arithmetic. This is exactly what `prototype/overlay.py` does:

```
x_pixel = int(x_normalised / 1000 * image_width)
y_pixel = int(y_normalised / 1000 * image_height)
```

Four ways to locate something

Pointing is only the first rung. Spatial models can locate things with increasing precision:



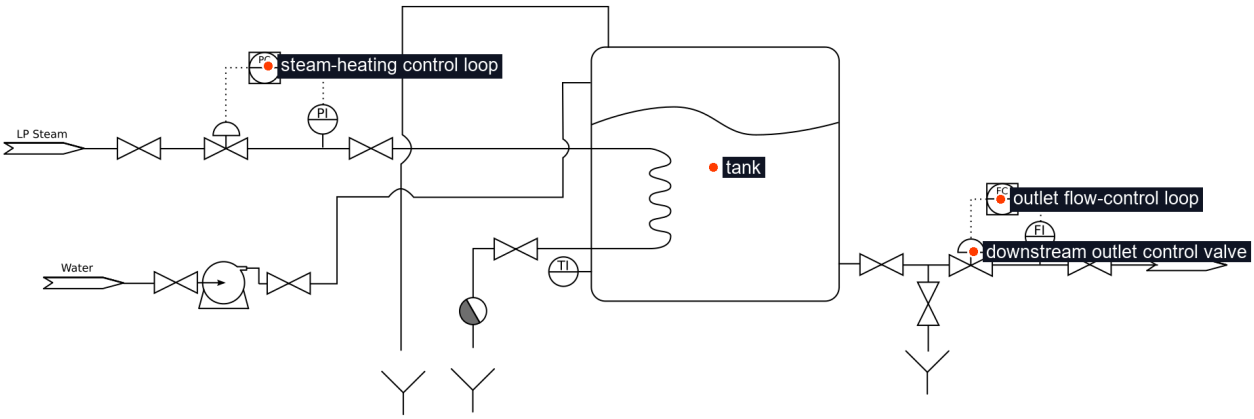
Method	Output	Good for	Documented for Gemini ER 2 [17]	Tested in this repo
Point	`[y, x]` + label	"Which one?" — identifying things	Yes	**Yes** — demonstrated
Box	`[ymin, xmin, ymax, xmax]` + label	Regions: a chart, a title block, a part	Yes	Not yet
Crop & re-inspect	A second, focused answer about a region	Tiny text, dials, dimensions	Yes (agentic code execution)	Not yet
Video / tracking	Points or events across frames	Progress, "when did it change?"	Yes	Not yet (and needs a privacy plan)

Each rung builds on the last. That's why the repository's own proposal scopes the next experiments as *box vs. point* and *crop and re-inspect*, and explicitly leaves video and robot orchestration out of scope for now.

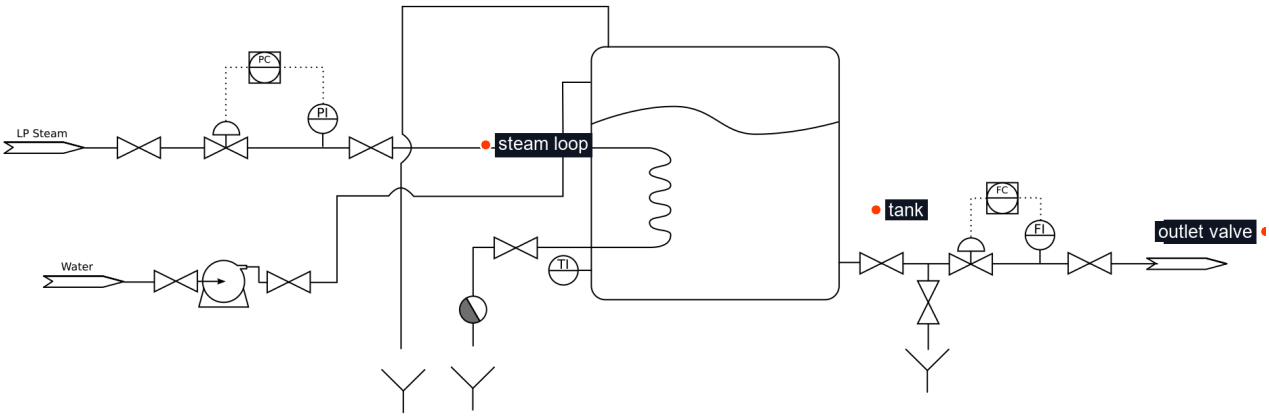
What we actually demonstrated

Here is a real output from this repository. Both models received the same public P&ID (a steam-heated tank with a flow-control loop) and the same prompt: point to the tank, the steam-heating control loop, and the outlet flow-control loop, and label the downstream outlet control valve.

Gemini Robotics-ER 2:



Azure AI Foundry `4Omni` (routed as `magma` in the demo; it is not Microsoft Magma):



Side-by-side version: `demo-results/public-pid-tank/comparison/gemini-er2-vs-foundry-4omni.jpg` .

Look closely at two labels, and at the raw numbers each model returned (saved in `demo-results/public-pid-tank/*/result.json`):

Label	Gemini ER 2 returned	Foundry 4Omni returned	What you see
outlet valve	`[595, 775]`	`[550, 1050]`	Gemini's dot sits on the valve after the flow controller. Foundry's x value is **1050 — outside the 0–1000 grid.** The overlay code clamps it to the image edge, which is why that dot hugs the right border.
outlet loop	`[474, 796]`	`[550, 1100]`	Same story: x = 1100 is off the grid.

Label	Gemini ER 2 returned	Foundry 4Omni returned	What you see
tank	`[400, 575]`	`[500, 700]`	Gemini's dot is inside the tank; Foundry's sits just outside its right wall.

Both models *described* the scene sensibly — the labels are all reasonable. Only one *located* it correctly **in this example**, and the other broke the coordinate contract outright. That is precisely the gap between describing and grounding — and a reminder to **validate that every coordinate is inside 0–1000** before trusting it.

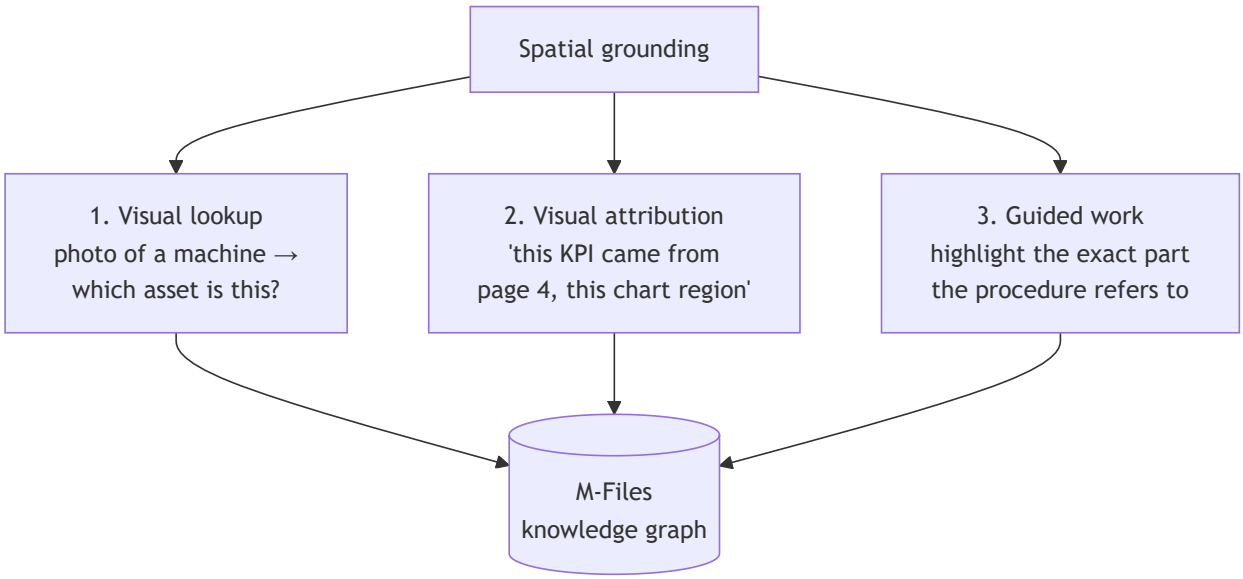
Now the honest part. This is one image. Across the repository's runs:

Run	Gemini ER 2	Foundry 4Omni
Six internal engineering drawings	Parseable points on 6 of 6	Parseable on 4 of 6, malformed on 2
Three simple public fixtures	Parseable points on all	Parseable points on all
Three complex public fixtures	Parseable on all (14–20 points)	Parseable on all (19–20 points)

"Parseable" means the model returned valid coordinates we could draw. It does **not** mean the points were correct. Nobody has yet labelled the drawings with the true locations and measured the error. Until that's done (Chapter 10), these are demonstrations, not benchmarks.

Why grounding matters for M-Files specifically

Grounding unlocks three things a text pipeline can't do:



1. **Visual lookup** — resolve a photo to the right object in the vault, even when the asset tag is worn or unreadable.

2. **Visual attribution** — every answer can point back to *where* on the page it came from. The repository's review of the metadata-poc design found no region-level attribution concept at all (Chapter 8).
3. **Guided work** — a procedure step says "replace the seal on flange B"; grounding highlights flange B on the photo.

Grounding vs. OCR bounding boxes

A fair question from a reviewer: "*Azure Document Intelligence already returns bounding boxes. Isn't that grounding?*"

It's a different kind of box.

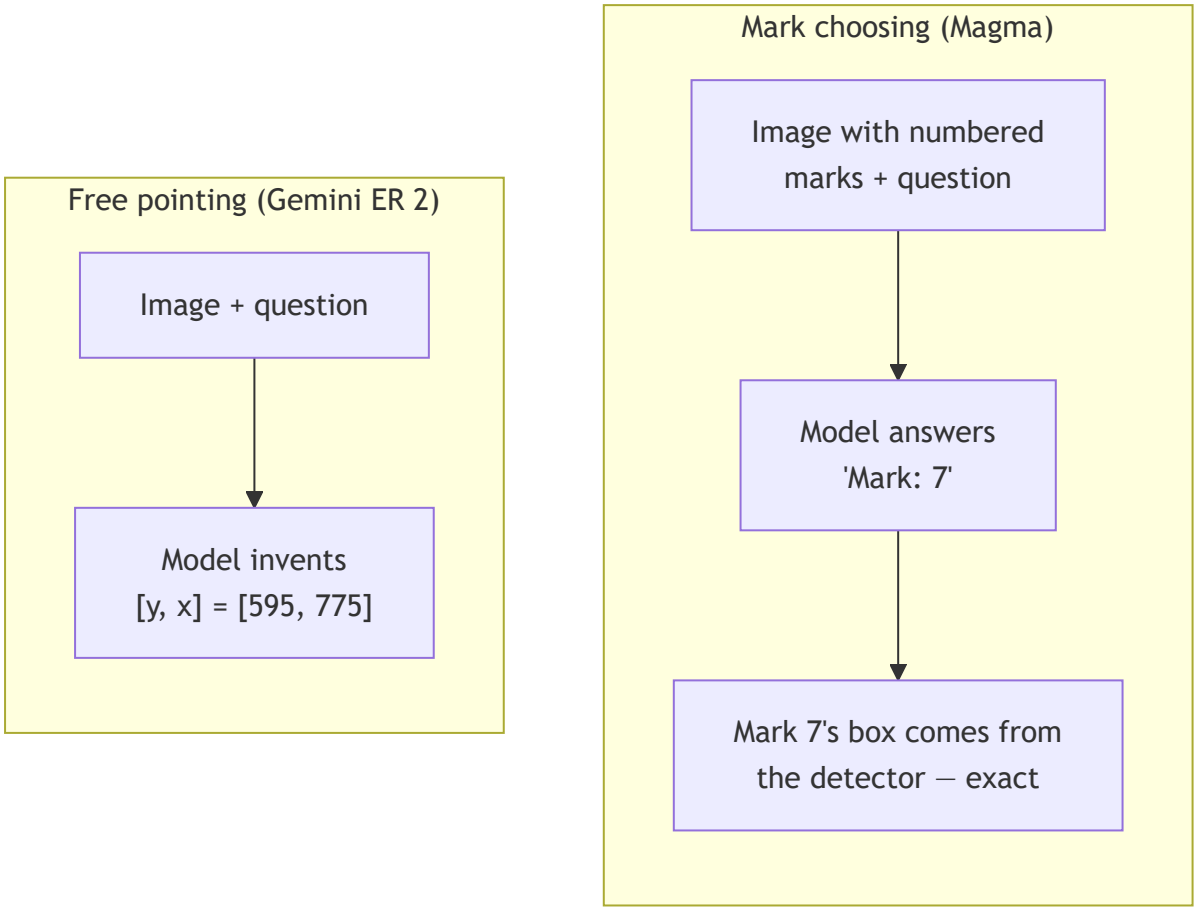
	OCR / layout boxes (e.g. Document Intelligence)	Semantic grounding (e.g. Gemini ER 2)
Boxes around	Text lines, words, table cells, figures	Whatever you asked about in plain language
Driven by	The page's structure	Your question
Can find a symbol with no text?	No	Yes
Deterministic?	Highly	No — it's a model's judgement
Best used for	Trustworthy text citation	Visual meaning and symbols

The two are complementary. The repository's own recommendation is to start Azure-native (Document Intelligence or Content Understanding as the extraction and citation foundation) and use semantic grounding for the visual cases those tools can't solve [18].

Another way to point: choosing a numbered mark

Back to the phone call. Instead of asking your colleague to find "the valve after the flow controller", imagine you'd both stuck numbered sticky notes on every valve beforehand. Now you just say: "*Close number 7.*" No ambiguity at all.

That's **Set-of-Mark (SoM)** grounding, the technique behind Microsoft's Magma model [21]. Something else — a detector, a layout tool, even a simple grid — draws numbered marks on candidate regions first. The model's job shrinks from "find the exact pixel" to "pick the right number".



	Free pointing	Choosing a mark
Who decides the location?	The model	The detector draws the candidates; the model chooses
Needs something upstream?	No	Yes — something must propose the marks
Coordinates as precise as…	The model's guess	The detector's box
Fails when…	The point drifts, or goes off the grid	The right thing never got a mark

Magma's paper shows the idea works: taking marks out of its training cut its ScreenSpot score from 61.4 to 57.4 and halved its robot-simulation score (35.4 to 17.7) [21]. For engineering drawings nobody has tested it yet, and the hard part is the upstream step — drawings have no built-in list of candidate regions the way a web page does. `docs/magma-readiness.md` sets out the options.

Two format traps if you ever parse both models' output: Magma writes `(x, y)` from 0 to 1; Gemini writes `[y, x]` from 0 to 1000. Same idea, opposite order, different scale.

The model landscape (as of September 2026)

Model	What it is	How to call it	Notes
Gemini Robotics-ER 2 (`gemini-robotics-er-2-preview`)	Google's embodied-reasoning VLM, built on Gemini 3.5 Flash	Gemini API	Preview; points, boxes, trajectories, video, agentic vision [17]
Gemini Robotics-ER 2 streaming	Low-latency variant	Gemini Live API	For continuous audio/video; out of scope for documents
Gemini Robotics 2	Google's vision-language-*action* model for robots	No public API model ID found	Robot control, not our use case (Chapter 6)
Azure Foundry general vision models	GPT-4o-class multimodal models	Azure AI Foundry	Inside Microsoft's boundary; weaker at precise pointing in our runs
Microsoft Magma-8B	Microsoft Research's vision-language-action model; answers by choosing numbered marks on the image	Listed in the Foundry catalog; Foundry Managed Compute (preview) if available in your project (needs GPU quota), or self-host (~17 GB GPU memory in bf16)	MIT licence; research use only per its model card; untested here — see `docs/magma-readiness.md`

Two practical warnings from Google's own documentation [17]:

- **Spatial outputs can be wrong or hallucinated**, especially on ambiguous or unusual inputs. Always draw the points and look.
- Google recommends **low "thinking" levels for simple pointing** and higher levels for counting or estimation — higher levels cost more and take longer.

Where M-Files stands

Demonstrated in a prototype; absent from the product. This repository proves that a spatial model can return usable point-and-label output on engineering drawings, and shows visually why precise pointing is harder than description. What's not done: boxes, crop-and-re-inspect, any accuracy measurement, and any connection to the M-Files vault. The model choice is also unresolved — the best grounding we've seen comes from a Google preview model that can't run inside Microsoft Foundry. Chapter 10 covers how to make that decision deliberately.

Chapter 5 — World models: imagining before acting

Catching a ball

Someone throws you a ball. You don't calculate a parabola. But you don't wait for the ball to arrive and then react either — you'd be far too late. Instead, a fraction of a second after it leaves their hand, your brain has already *imagined* where it will be, and your hand moves there.

You carry a small, fast simulation of the world in your head. You use it to predict what happens next — and what happens *if you act*. If I reach left, the ball lands in my palm. If I don't, it hits my chest.

That internal simulation is what AI researchers call a **world model**.

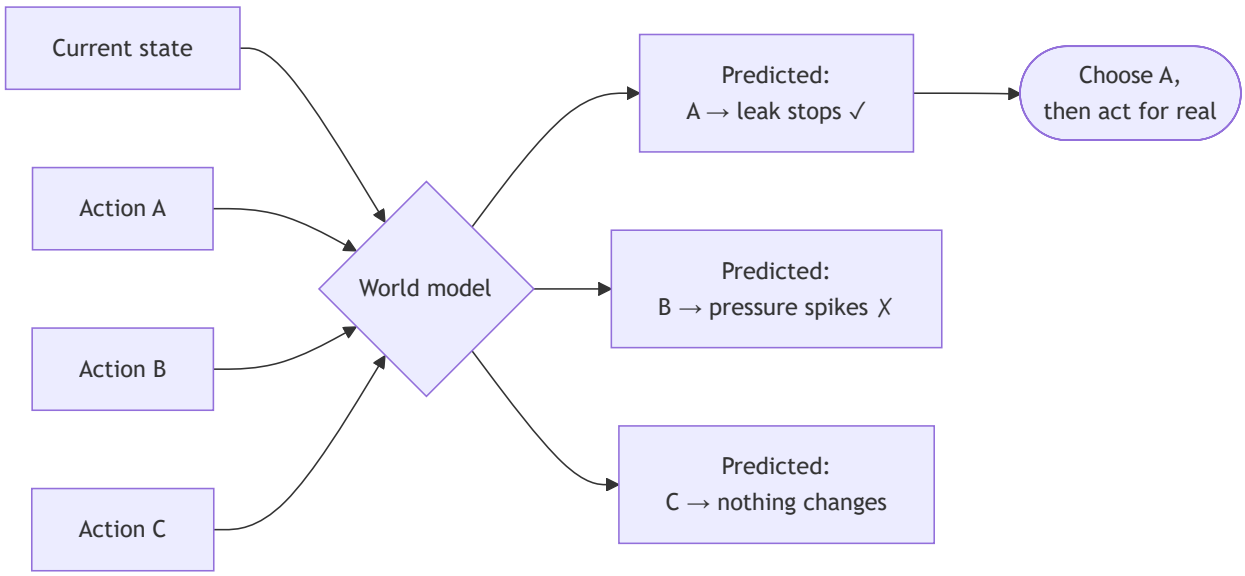
The formal definition

To formalise: **a world model is a model that predicts how the state of the world will change, given the current state and an action.**

In one line:

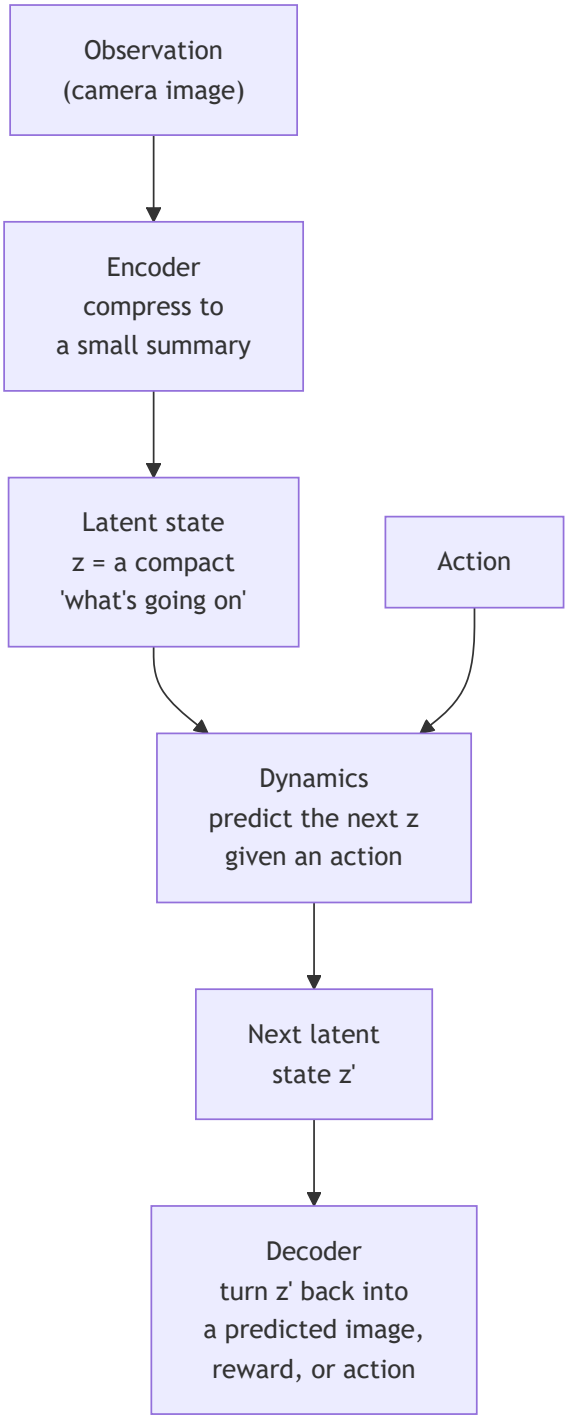
```
next_state = world_model(current_state, action)
```

If you have one, you can do something very powerful: **try actions in your imagination before trying them in reality.** That's planning.



How a learned world model works

Modern world models are learned from data — usually video plus actions. They have three parts, and the names are worth knowing because every paper uses them:

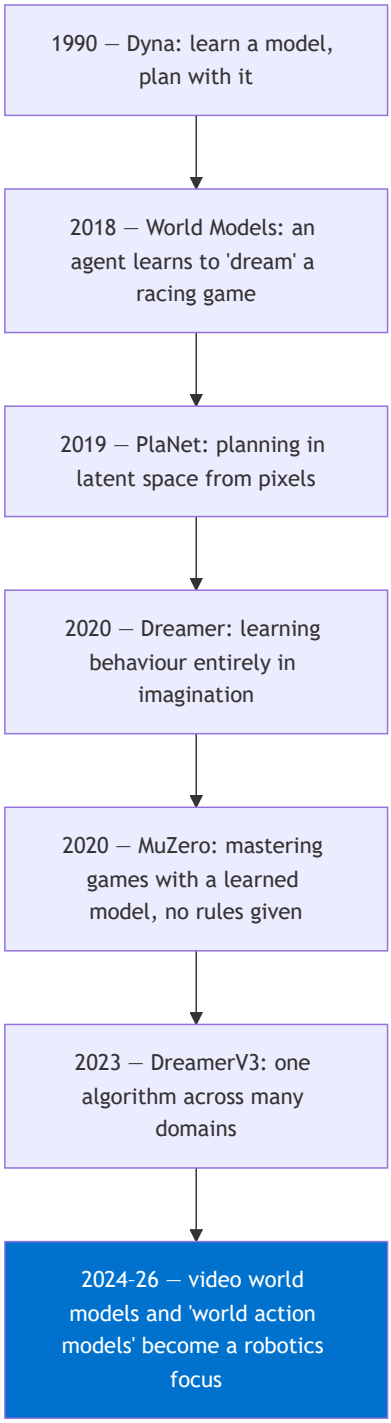


1. **Encoder** — squeezes a huge observation (millions of pixels) into a small list of numbers, the *latent state*. Think of it as the model's private shorthand: "arm near cup, cup upright, table clear."
2. **Dynamics** — the heart of it. Given the latent state and an action, predict the next latent state.

3. **Decoder** — optionally turns a predicted latent state back into something checkable: a predicted image, a reward, or an action.

Why not predict pixels directly? Because predicting every pixel of the next frame is enormously expensive and mostly wasted — you don't care about the exact texture of the wall. Predicting in the compact latent space is far cheaper.

A short history



NVIDIA's Jim Fan argues that the successor to today's vision-language-action models will be **world action models** — models that learn physics from vast amounts of cheap video, rather than spending most of their capacity on language [1].

Three things a world model is **not**

This is where most confusion lives.

Claim	Reality
"An LLM is a world model."	An LLM predicts the next **word**. It knows a lot *about* the world from text, but it doesn't predict the next **physical state** after an action. Language knowledge ≠ physics.
"A VLA is a world model."	A VLA (Chapter 6) maps observation → action. It may never explicitly predict what happens next. Some do; many don't.
"A good video generator is a world model."	Generating plausible-looking video isn't the same as predicting correctly *what your action will cause*. Pretty ≠ right.

The evaluation question to always ask: **does the model predict the consequences of actions, and how well does that prediction hold up when conditions change?**

Model-free vs. model-based

There are two broad ways to learn to act. This comparison appears in almost every robotics paper:

	Model-free	Model-based (world model)
Idea	Learn "in this situation, do this" directly by trial and error	Learn how the world works, then plan
Analogy	Learning to ride a bike by falling a thousand times	Studying how balance works, then practising in your head
Data needed	Lots	Less (imagination is free)
Weakness	Data-hungry, brittle when things change	Errors compound — small prediction mistakes snowball over long horizons
Examples	PPO, SAC	Dreamer, MuZero, TD-MPC

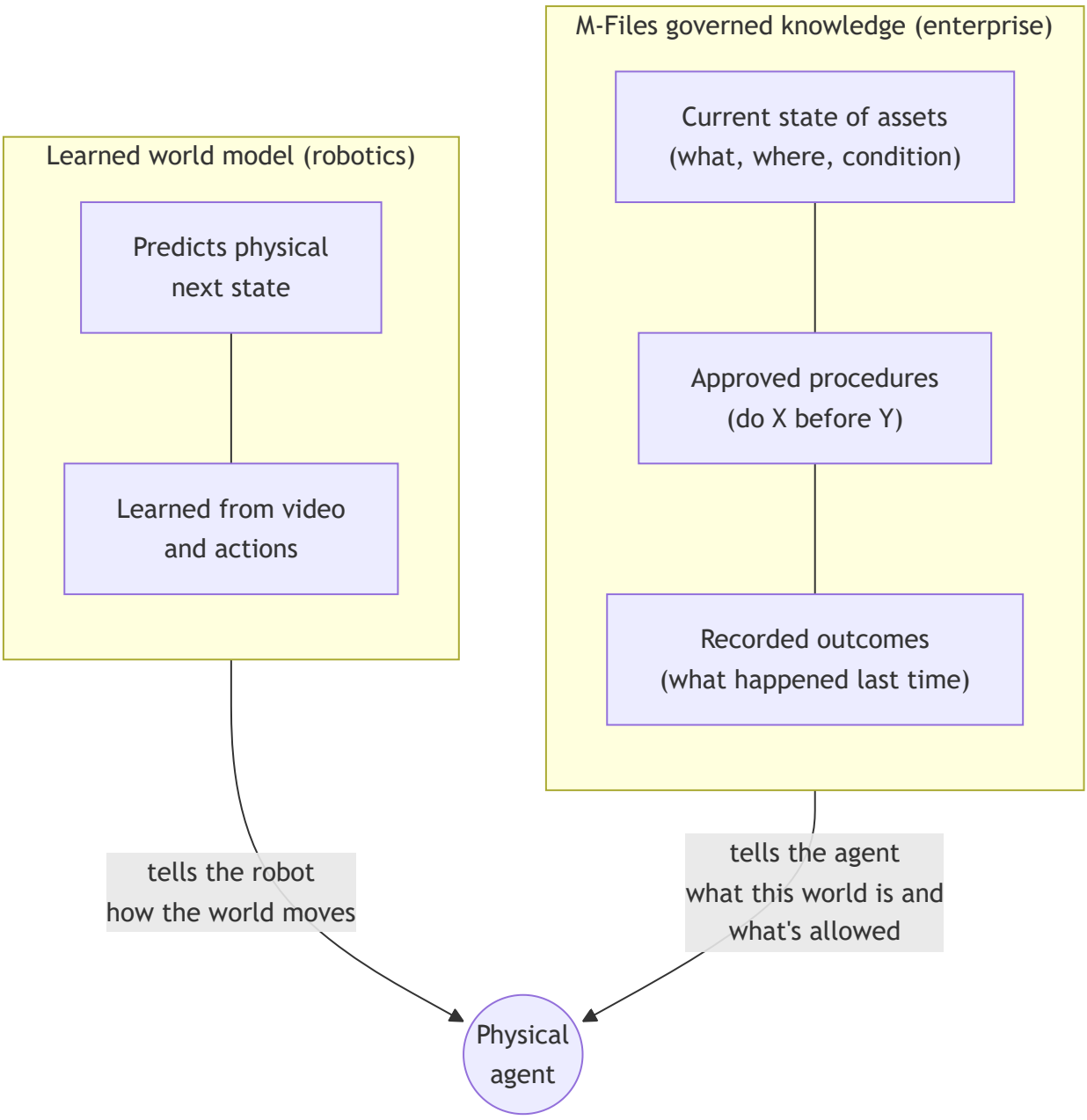
That last weakness — **compounding error** — is why world models are powerful for short horizons and still risky for long ones.

An enterprise way to think about it

Here's a bridge that's useful for M-Files conversations, and it needs to be stated carefully.

A learned world model predicts *physics*: push the cup, it slides. An enterprise doesn't need M-Files to predict physics. But it does keep a very important kind of structured world knowledge:

- **The current state of the world** — this pump, serial number X, installed on line 3, last serviced in March.
- **Approved expectations about actions** — the maintenance procedure says: close valve V2 *before* removing the seal, or the line will depressurise.
- **The history of what actually happened** — every past repair and its outcome.



These are **not the same thing**, and it would be wrong to call a knowledge graph a world model in the technical sense. But they're complementary halves of what a physical agent needs: one says *how things move*, the other says *what this specific thing is, what state it's in, and what we've agreed is the right way to act on it.* The robotics industry is racing on the first half. The second half is where M-Files lives.

Where M-Files stands

Not our job to build — but important to understand. M-Files should never train a learned world model; Google, NVIDIA and others are investing billions there. What M-Files should do is be fluent enough in the concept to position its governed knowledge as the complement: the enterprise's record of asset state, approved procedures, and outcomes. When someone asks "is M-Files building a world model?", the honest answer is: "No — we're building the governed memory a world model's agent consults before it acts."

Chapter 6 — From seeing to doing: how robots learn to act

Learning to cook by watching

How did you learn to crack an egg? Probably not from a textbook. Someone showed you — maybe a few times — and then you tried, made a mess, and tried again.

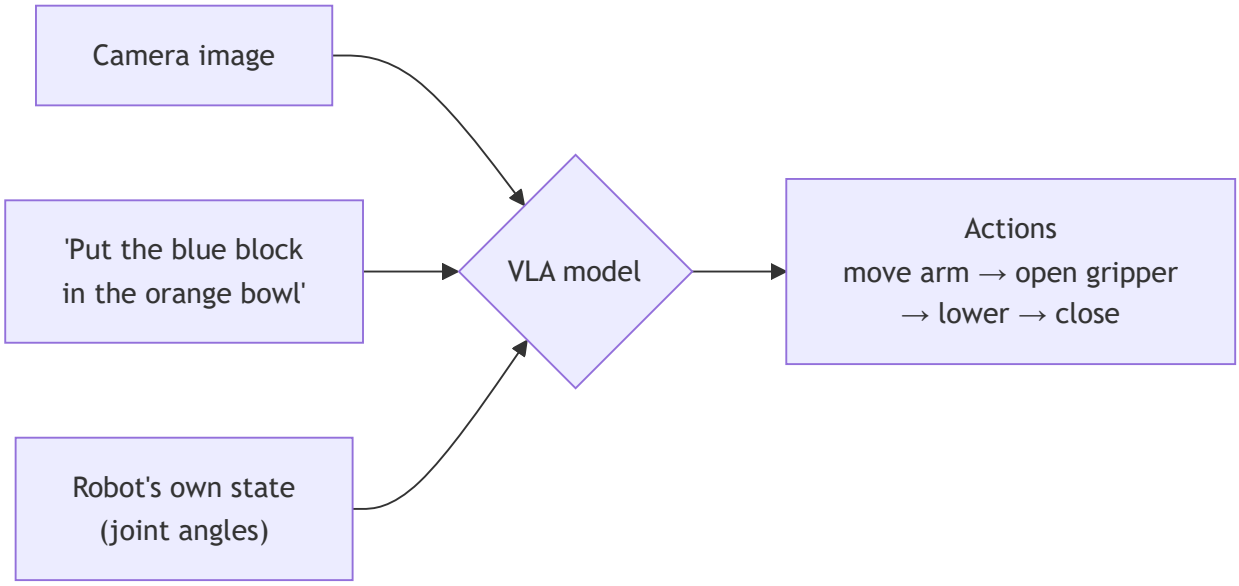
Robots learn in the same two ways:

1. **Watching** — copying demonstrations (imitation learning).
2. **Trying** — trial and error with a reward (reinforcement learning).

Modern robot "brains" combine both, plus a third ingredient that changed everything: **pre-training on the internet**, so the robot already knows what an egg *is* before it ever sees one.

The key term: VLA

To formalise: **a vision-language-action model (VLA) takes camera images and a plain-language instruction, and outputs robot actions.**



It's a VLM (Chapter 3) with an extra output: instead of only answering in words, it answers in motor commands.

The family tree:

Year	Model	What was new
2022	**RT-1** (Google)	One transformer trained on lots of real robot demos, many tasks
2023	**RT-2** (Google)	Started from a web-trained VLM — the robot inherits internet knowledge
2023–24	**Open X-Embodiment**	20+ labs pooled robot data across many robot types
2024	**OpenVLA**, **Octo**	Open-source VLAs anyone can fine-tune
2025–26	**Gemini Robotics 2**, **π-series**, others	General-purpose policies, whole-body control, multi-robot coordination

---

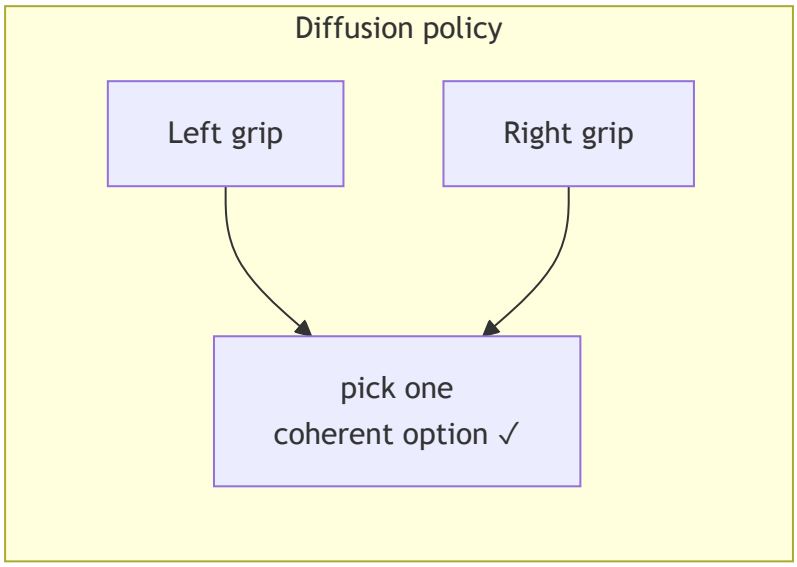
Two tricks that made VLAs work

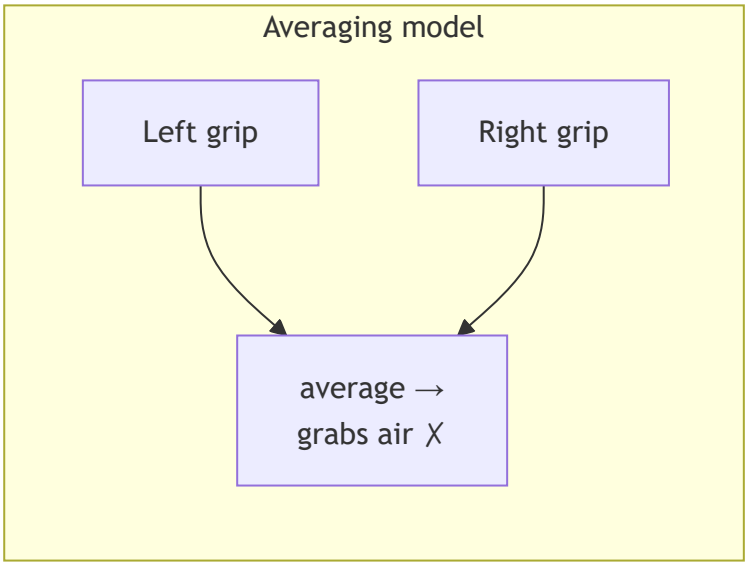
Trick 1 — Action chunks

Instead of predicting one tiny motor command at a time (jerky, slow), models predict a short *chunk* of future actions at once — say, the next half-second of movement. Smoother, faster, and fewer chances to wobble.

Trick 2 — Diffusion policies

Often there are several good ways to do a task — grab the cup from the left or the right. A normal model averages them and grabs the air in the middle. **Diffusion policies** (borrowing the technique from image generators) produce a *distribution* of possible action sequences and pick one coherent option.

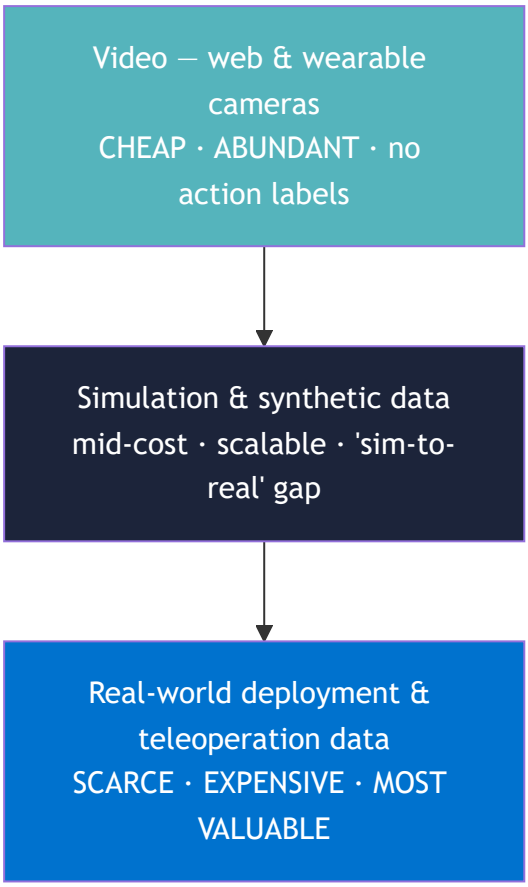




The data problem: the data pyramid

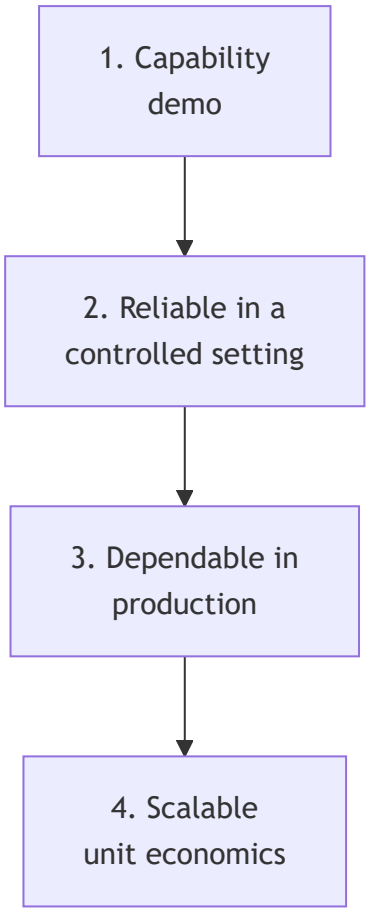
Language models were trained on the whole internet. Robots don't have an internet of robot data. This is the central bottleneck of the field.

Two independent sources describe the same shape. Jim Fan (NVIDIA) calls it a data ladder [1]; McKinsey calls it a data pyramid [5]:



- **Bottom:** human video is everywhere and nearly free, but it shows no robot actions.
- **Middle:** simulation can generate unlimited data, but simulated physics never perfectly matches reality.
- **Top:** real robot data is gold, and painfully slow to collect.

McKinsey adds a reliability ladder that describes how any physical-AI system matures [5]:



Most pilots die between steps 2 and 4. Keep this ladder in mind — it applies to our work too (Chapter 11).

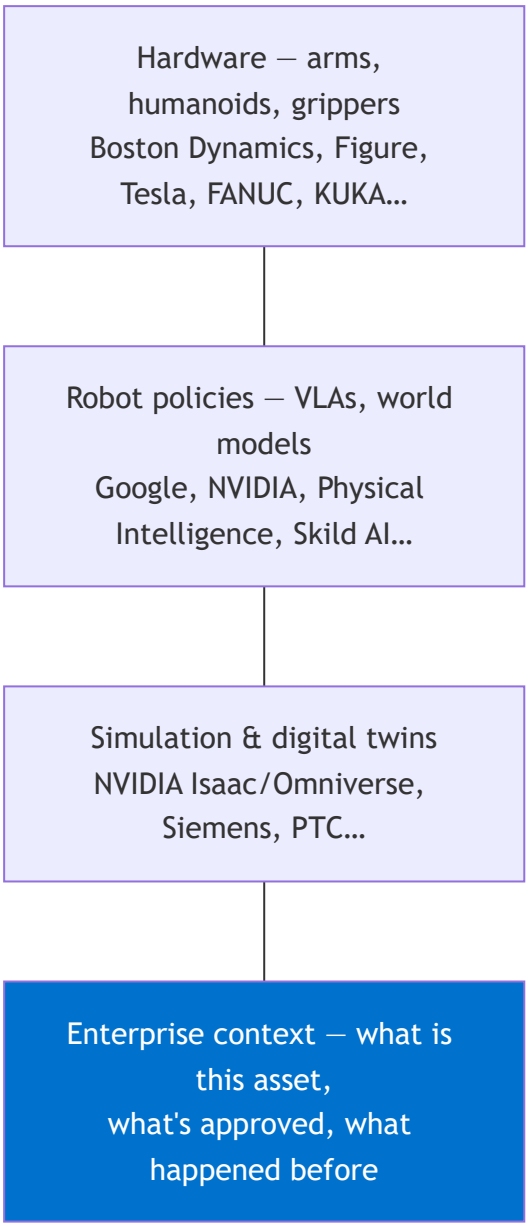
Where reinforcement learning fits (briefly)

Reinforcement learning (RL) is learning from reward: try something, get a score, adjust. It produced famous results in games and simulated robots (DQN, PPO, SAC). In real-world robotics it's usually used to *fine-tune* a policy that was first learned by imitation, because pure trial-and-error on real hardware is slow and can break things.

You don't need to master RL for VMAN Vision. You need to recognise the words and know that "the robot learned it with RL" usually means "in simulation, then carefully transferred" — which is exactly where the sim-to-real gap bites.

What this means for the industry — and for us

The robotics race is being run on these layers:



Every layer above the last one is crowded and extremely well-funded. The last layer — enterprise context — is where a robot or a technician finds out *what this specific machine is and what the approved procedure is*. That's the layer this book keeps returning to.

Where M-Files stands

Deliberately out of scope — correctly. M-Files has no robot training data, no robotics team, and no reason to acquire either. The Manufacturing Blueprint's own words set the boundary: M-Files "records outcomes and recommends actions; it does not execute transactions" [10]. Understanding VLAs matters for two reasons only: to speak credibly with robotics partners, and to recognise that every VLA-driven robot in a factory still needs someone to tell it which machine is which and what the approved procedure is.

Chapter 7 — Knowledge graphs: our secret weapon

The veteran technician

Every plant has one. The person who's been there twenty years. You ask them about a pump and they say:

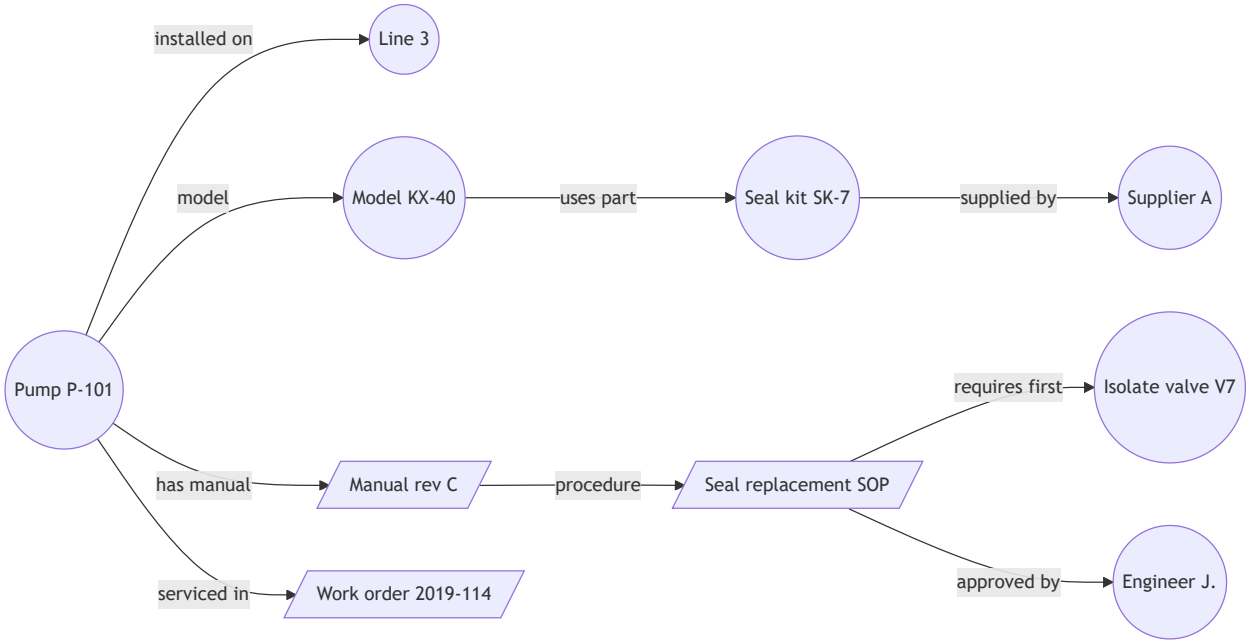
"That's the one we replaced in 2019 after the flood. It's the same model as the one on line 2, but the seal kit is different — the supplier changed the part number. And don't forget to isolate V7 first, the drawing's wrong about that."

Notice what they're doing. They don't recite a document. They **follow connections**: pump → replacement history → sister unit → supplier → part number → procedure → known exception. Their knowledge is a web, not a pile.

That web is a **knowledge graph**. And when that person retires, it's the thing the company most fears losing.

The formal definition

To formalise: **a knowledge graph stores things (nodes) and the named relationships between them (edges), so that knowledge can be followed, queried, and reasoned over — not just searched.**



Every arrow is something a search engine can't easily answer but a graph can: *"Which pumps use seal kits from Supplier A?" "What must be isolated before replacing this seal?" "Who approved this procedure?"*

Knowledge graph vs. folders vs. vector database

Three ways to organise the same knowledge. This comparison is worth memorising — it comes up in every architecture discussion.

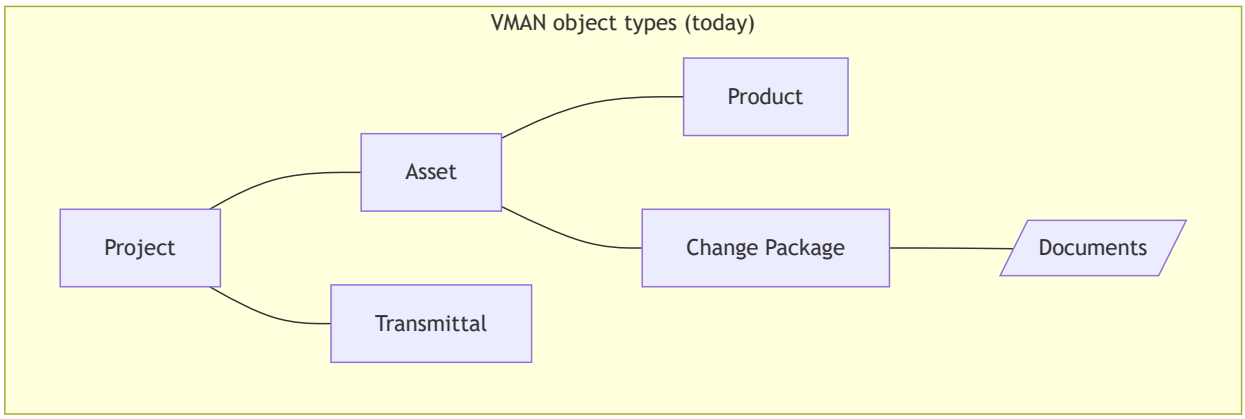
	Folders	Vector database	Knowledge graph
Structure	A tree: one place per file	A cloud of points by meaning	A web of named relationships
Good question	"Where's the Line 3 folder?"	"Find text similar to 'seal leak'"	"What must I isolate before this repair?"
Strength	Simple	Fuzzy, semantic search	Precise, explainable, multi-hop
Weakness	A file lives in only one place	Can't follow relationships; can't say *why*	Needs structure to be defined
Explains its answer?	—	Rarely	Yes — the path *is* the explanation

The modern answer is **both**: vector search to find candidates, a graph to follow the relationships and explain the result. This pattern is often called **GraphRAG** (Chapter 8).

M-Files is already shaped like a graph

M-Files doesn't organise by folders. It organises by **objects with metadata**, and **relationships between objects**. Our Product Strategy 2025–26 names this directly — relationships across "documents, projects, people, machines, companies" — and notes that "knowledge graphs are a favored format for AI" [9].

In VMAN, the manufacturing vertical, the object types are already graph-like: Projects, Assets, Products, Transmittals, Change Packages — all linked.

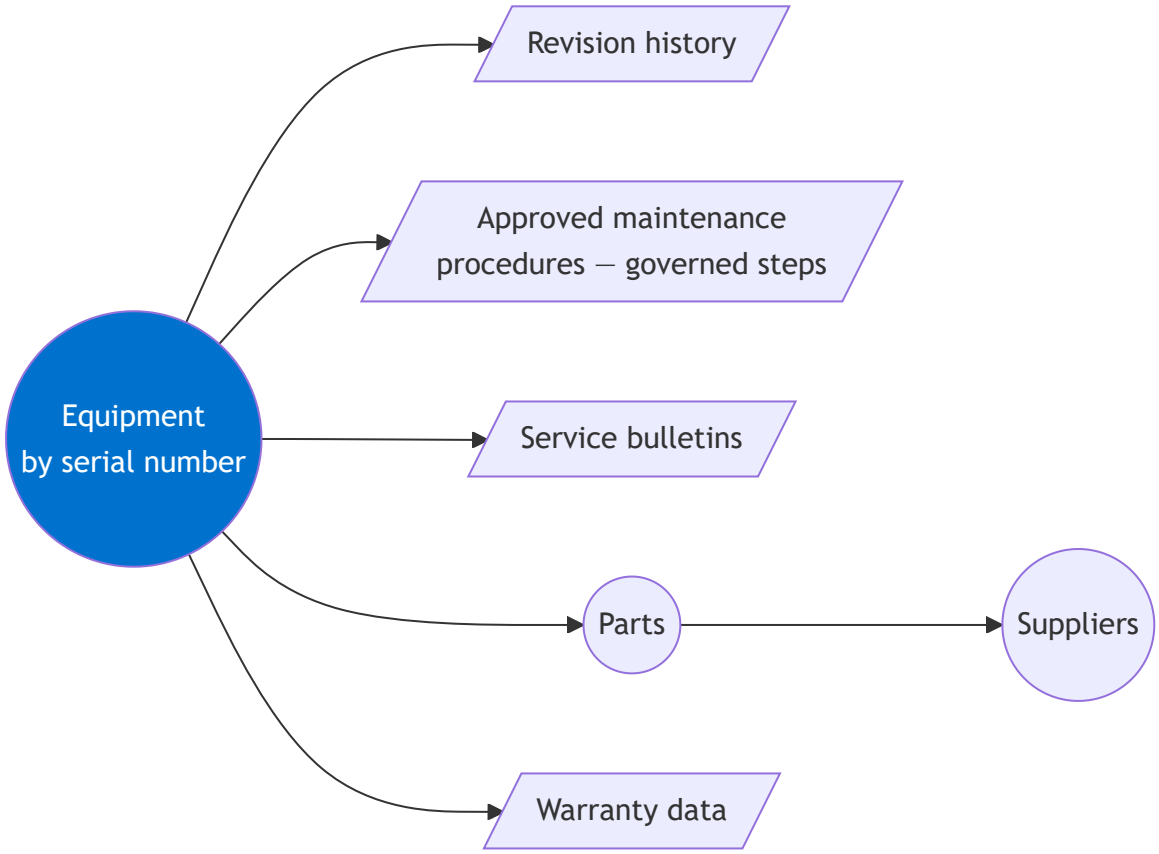


The piece that's planned, not built: Installed Base & Maintenance Knowledge

The Manufacturing Blueprint describes exactly the graph a physical agent would need, and lists it as **planned** [10]:

> Equipment by serial number … approved maintenance procedures as governed steps an assistant renders verbatim.

The 2027 forward-looking thoughts call it "validated design work, not yet built" [11]. As described, it links:



Look at that diagram next to the veteran technician's knowledge at the start of this chapter. It's the same web.

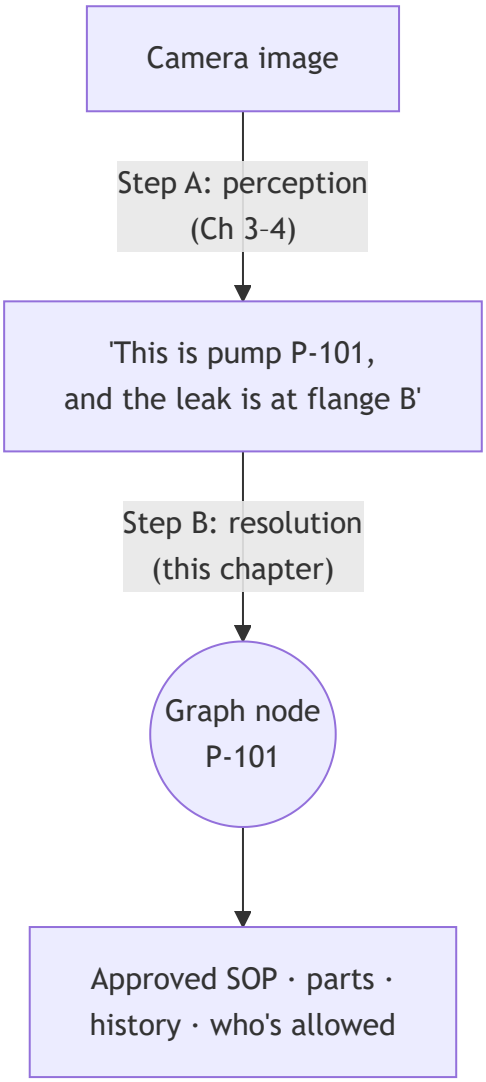
Two phrases in the Blueprint deserve attention:

- **"governed steps an assistant renders verbatim"** — the procedure is not paraphrased by an AI. It's the approved text, shown exactly. That's what makes it safe to put in front of a technician, or a robot.

- **"your systems keep the transaction, M-Files keeps the governed context"** [10] — M-Files isn't replacing the ERP or the maintenance system. It holds the *meaning and the approvals* around them.

Why a physical agent needs this graph

Go back to the goal from Chapter 1: a camera sees a machine, and the system returns the approved procedure. There are two hard steps:



- **Step A — perception** turns pixels into "pump, flange B". That's a vision problem.
- **Step B — resolution** turns "pump" into *the specific node in the graph*: serial number, line, manual revision, permissions. That's a graph problem.

Step B is where most AI demos quietly cheat — they stop at "it's a pump". An enterprise needs "it's *this* pump, and here's *its* approved procedure." Without the graph, the camera sees a pump. With the graph, the agent knows which one.

Today, resolution usually depends on a readable asset tag or serial plate — and the Integration Roadmap scopes tag-to-document linkage for exactly this [12]. The gap vision fills is the case where the tag is worn, painted over, or simply not in view.

How the giants are doing the same thing

We're not the only ones who noticed that physical AI needs a graph:

Company	Their graph	What they're doing with it
SAP	SAP Knowledge Graph (Sapphire 2026)	Grounding agents in business metadata; a Tetra Pak lab PoC had a humanoid run SAP warehouse workflows guided by that context [13]
Palantir	Ontology	Models Factory → Line → Machine → Part with live telemetry for maintenance and planning [14]
Siemens	Digital twins + knowledge graphs	Structures relationships across plant data for AI agents [15]

Notice the pattern: each of them grounds the agent in **structured, transactional data** — ERP records, sensor telemetry, CAD geometry. Here is where M-Files is different. A great deal of a plant's real knowledge doesn't live in those systems: it lives in manuals, marked-up drawings, SOPs, service bulletins, inspection photos, and approvals — **documents**. That's the content M-Files governs.

> ### Where M-Files stands
>
> **Our strongest card — but half-played.** M-Files already is a graph of documents, people, machines and projects, with governance built in. That is genuinely the memory a physical agent needs, and it covers the document-shaped knowledge the ERP-centred giants handle least well. But the specific graph that matters most — Installed Base & Maintenance Knowledge — is planned, not built, and the engineering-knowledge-graph milestone is H1 2028 [12]. The single most valuable thing this workstream can do is give that planned module a reason to exist sooner: a working demonstration that a camera can resolve to it.

Chapter 8 — Retrieval that can see

The open-book exam

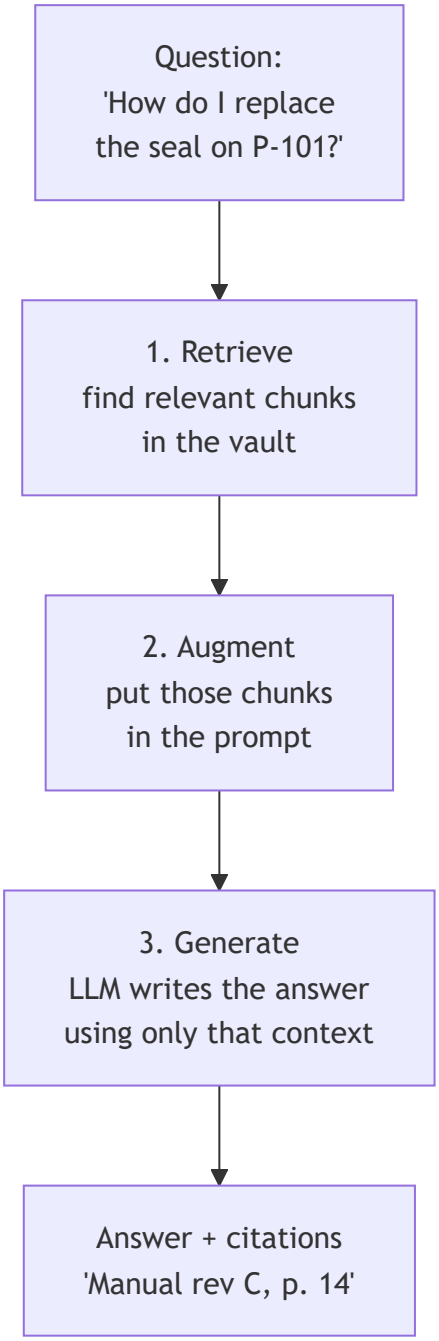
Imagine two students sitting an exam. The first has to answer from memory. The second is allowed to bring a textbook, look things up, and — crucially — **cite the page** they got each answer from.

Who do you trust more? The second one, every time. Not because they're smarter, but because you can **check** them.

That's the idea behind **retrieval-augmented generation (RAG)**, and it's why nearly every enterprise AI system is built on it.

RAG in one picture

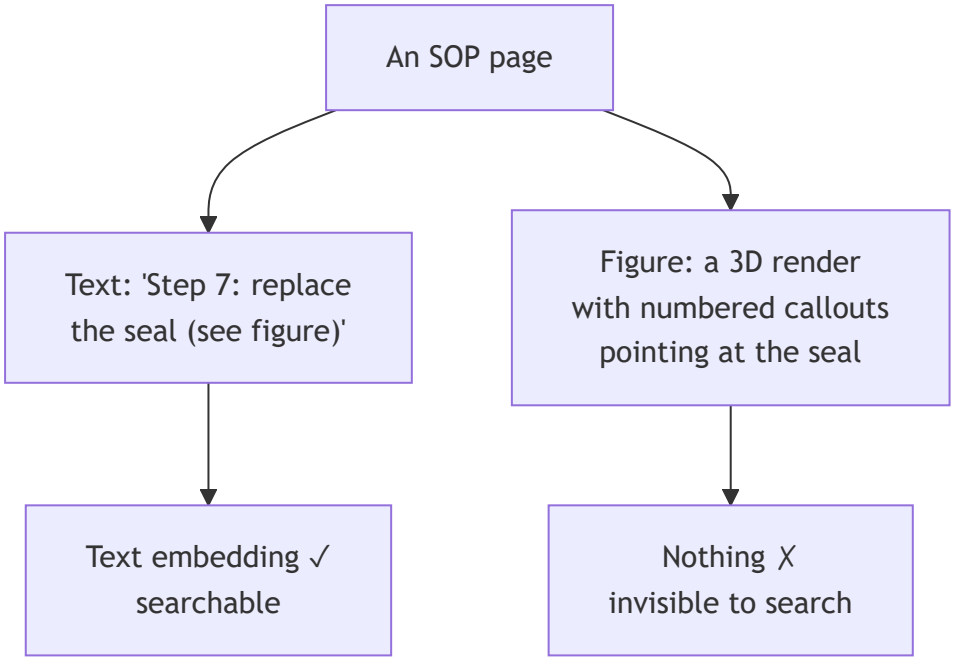
To formalise: **RAG is a pattern where a system first retrieves relevant pieces of your own content, then gives them to a language model to write an answer grounded in that content.**



The retrieval step usually works like this: every document is split into chunks, each chunk is turned into an **embedding** (a list of numbers representing its meaning — Chapter 3's "meaning space"), and stored in a vector index. A question becomes an embedding too, and the index returns the nearest chunks.

The blind spot: text-only retrieval

Here's the problem. If your embeddings are built from **text**, then anything that isn't text is invisible to search.



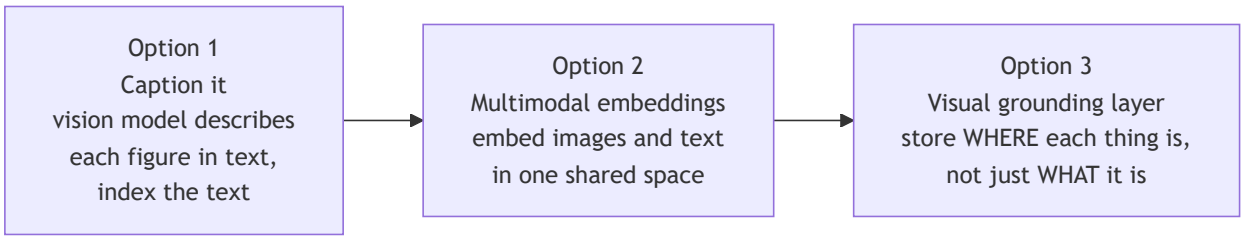
The answer to "*which part is the seal?*" is in the figure — and the figure was never indexed. For engineering content, that's often the most important part of the page.

Two specific facts from our own platform:

- **Aino Search is built on text embeddings** (Azure AI Search).
- **Aino's "Images" processing mode** sends page images to the LLM only when files are ≤10 MB; larger files silently fall back to text only (per the AI-Metadata team's workflow documentation, reviewed 2026-09-08). Image-heavy engineering documents are exactly the ones that exceed that.

Three ways to make retrieval see

There's a ladder of options, from cheapest to most capable:

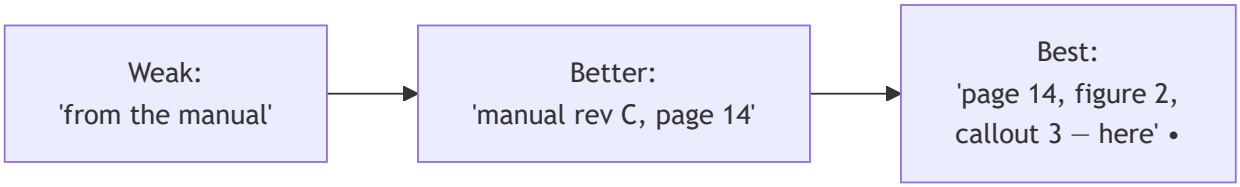


	1. Caption, then index	2. Multimodal embeddings	3. Visual grounding layer
How	A VLM writes a text description of each image; index that	Images and text embedded in the same space (CLIP-style)	Store labelled points/boxes per page alongside the text
Finds	Images whose *description* matches	Images whose *content* matches	The exact *region* on the page
Can cite a region?	No	No	**Yes**
Azure-native path	Azure Content Understanding can generate figure descriptions [18]	Multimodal search in Azure AI Search [18]	Not available off the shelf
Cost	Low	Medium	Higher

Options 1 and 2 can be built from Azure services today. Option 3 is where this repository's visual-grounding work contributes — and it's the only option that answers *"show me where"*.

Source attribution: "show me where"

Go back to the open-book exam. Citing "the textbook" is weak. Citing "page 14" is good. Citing **"page 14, the diagram in the top-right, callout 3"** is what a technician actually needs.



Where do these citation levels come from today?

- **Text-level citations** — the sibling repository `complex-document-grounding-poc` has an Azure Document Intelligence adapter that preserves page numbers, text spans and bounding regions for every observation (implemented and offline-tested; live baseline pending).
- **Region-level visual citations** — the review of the AI-Metadata team's design documents found **no** concept of citing back to a location on a page. This is an open gap on both sides.

Visual grounding (Chapter 4) is structurally suited to the "best" level: a point or box on the page *is* a citation.

RAG vs. Agentic RAG vs. GraphRAG

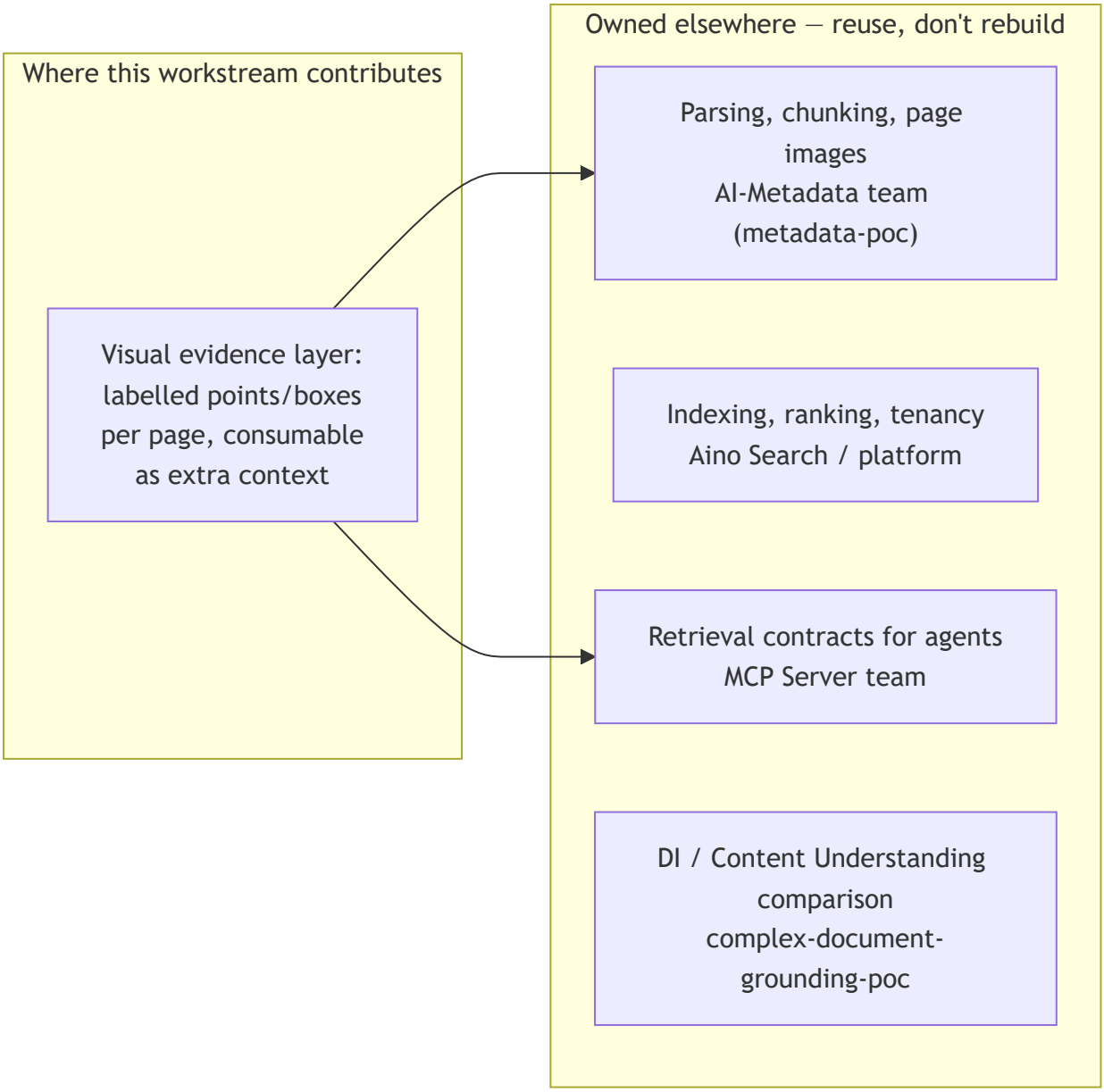
Three variations you'll hear constantly:

	Classic RAG	GraphRAG	Agentic RAG
Retrieval	One search, top-k chunks	Search, then follow graph relationships	An agent decides what to search, when, and whether to search again
Good for	"What does the manual say about X?"	"What else is connected to X?"	Multi-step questions needing several lookups
Example	Find the seal procedure	Find the procedure *and* the parts *and* the supplier	"Is this pump's warranty still valid, and if not, who supplies the seal?"
Fit for M-Files	Today's Aino Search	Natural fit for the metadata graph (Chapter 7)	Natural fit for M-Files agents and MCP (Chapter 9)

For a physical agent, the natural end state is **agentic GraphRAG over multimodal content**: an agent that can look at a photo, find the right node in the graph, follow its relationships, and cite exactly where each fact came from.

Who owns what (don't build a parallel pipeline)

Retrieval at M-Files is not a blank page. Other teams own large pieces of it, and the repository's own guidance is clear about not duplicating them:



A Q4 planning item — Complex Document Retrieval and Foundation — exists to align the MCP, metadata-agent and platform teams on exactly this boundary.

Where M-Files stands

Text-strong, picture-blind. Our search reads words well and ignores what's in figures and drawings — which, for manufacturing content, is often the most important part. Azure offers building blocks for captioning and multimodal search that could be adopted without leaving Microsoft's boundary. What nobody at M-Files has yet is region-level attribution — "it came from here, on this page." That's the capability this workstream's grounding layer is shaped to add, *as input to* the teams who own retrieval, not as a rival to them.

Chapter 9 — Agents, tools and MCP: giving the brain hands

The brilliant consultant with no phone

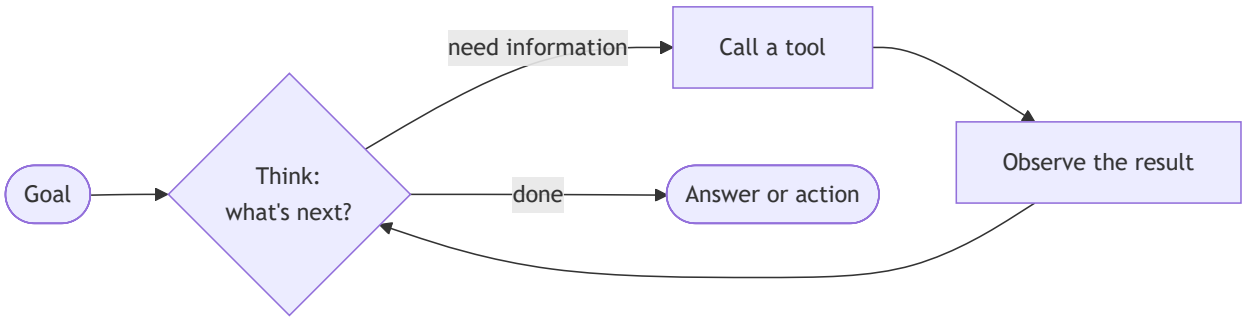
Imagine hiring the smartest consultant in the world, then locking them in a room with no phone, no computer, and no access to your systems. They can give you great general advice. But ask *"what's the stock level of seal kit SK-7?"* and they can only guess.

Now give them a phone and a list of numbers to call: the warehouse, the maintenance system, the document vault. Suddenly they can **look things up and get things done**.

A language model on its own is the consultant in the locked room. **Tools** are the phone. An **agent** is the consultant who knows when to pick up the phone, who to call, and what to do with the answer.

What an agent is

To formalise: **an AI agent is a system in which a language model repeatedly decides what to do next — call a tool, look at the result, decide again — until the goal is met.**



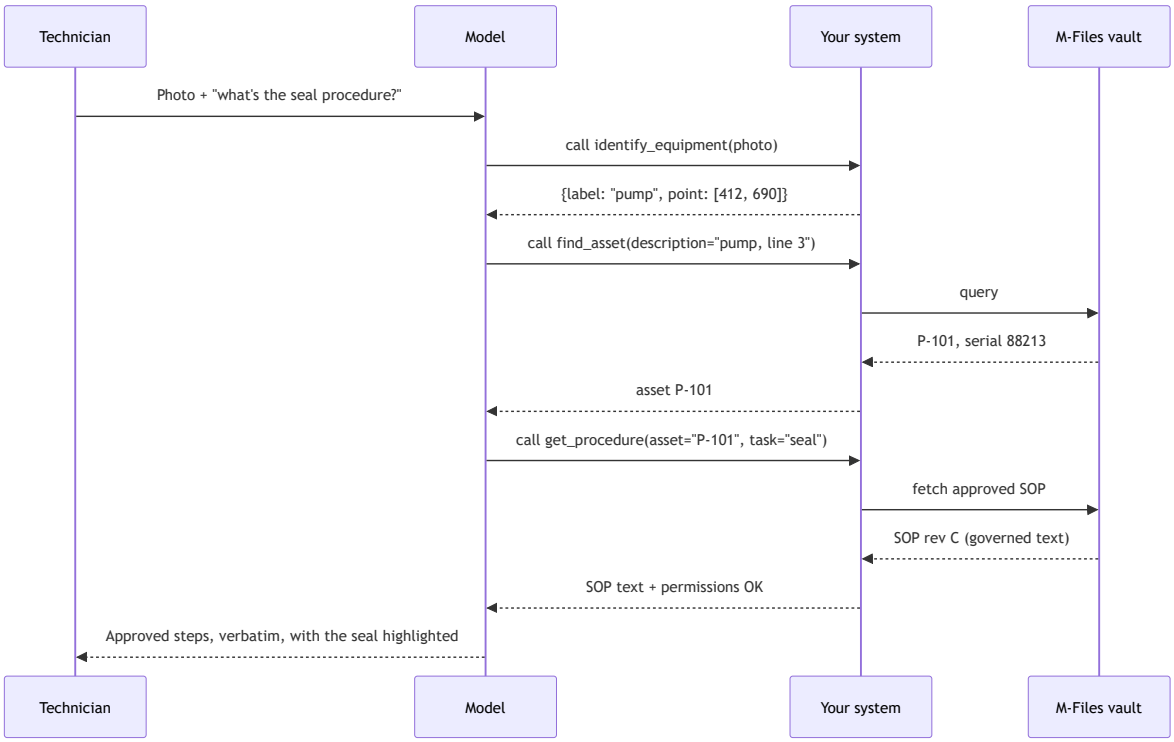
This loop — think, act, observe, repeat — is often called **ReAct** (Reason + Act). It's the same sense → plan → act → check loop from Chapter 2, just running over software systems instead of the physical world.

Tool calling, concretely

A "tool" is just a function the model is allowed to request, described to it in a structured way:

```
{
  "name": "find_asset",
  "description": "Find an asset in the vault by serial number or description.",
  "parameters": {
    "serial_number": "string (optional)",
    "description": "string (optional)"
  }
}
```

The model never runs code itself. It *asks* for a tool call; your system runs it and returns the result:

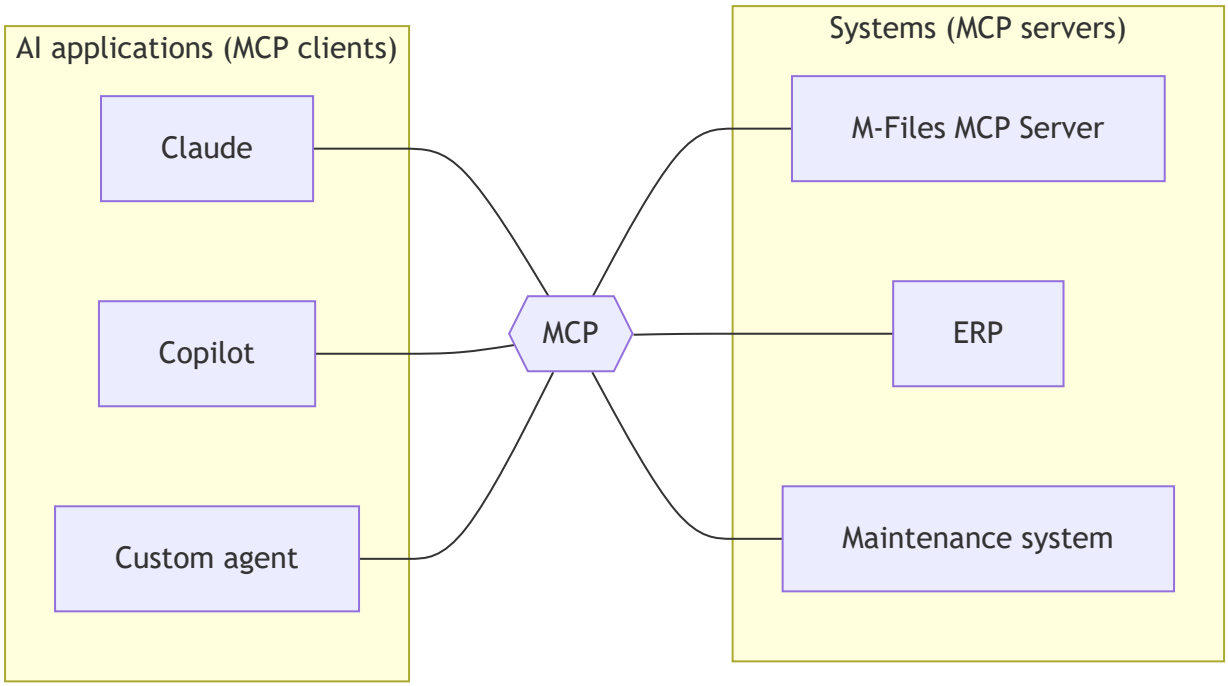


Gemini Robotics-ER 2's own quickstart demonstrates exactly this loop — a stateful sequence of function calls, each result fed back to the model — using a mock robot arm [17]. Swap the mock robot functions for vault lookups and it's the diagram above.

Why MCP exists

Every AI application used to wire up its own tools in its own way. Ten apps × ten systems = a hundred custom integrations.

The **Model Context Protocol (MCP)** is a standard plug. A system exposes its tools once, as an **MCP server**; any MCP-capable AI application (**MCP client**) can use them.



To formalise: **MCP is an open protocol that standardises how AI applications discover and call tools, read resources, and use prompts exposed by external systems.**

An MCP server offers three kinds of things:

Primitive	What it is	M-Files example
Tools	Actions the model can request	Search the vault, fetch an object, read metadata
Resources	Content the model can read	A document, a procedure
Prompts	Reusable instruction templates	"Summarise this change package"

API vs. MCP vs. function calling

People mix these up constantly.

	Plain API	Function calling	MCP
What it is	A system's interface for programs	A model's ability to request a function	A standard for exposing tools to *any* model app
Who uses it	Developers	One model, inside one app	Any MCP client
Reuse across AI apps	Build an integration per app	Re-describe tools per app	Build once, reuse everywhere

	Plain API	Function calling	MCP
Relationship	MCP servers usually wrap APIs	MCP tools are called *via* function calling	The standard on top

---

The M-Files MCP Server

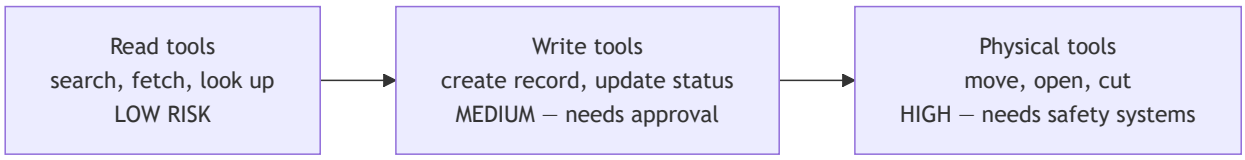
M-Files has an MCP Server in private beta, letting AI assistants search and read vault content. Its beta page lists a tracked limitation: **"Image reading is not available in Beta."**

For a physical-agent workflow, that one line matters a lot. In the sequence diagram above, the very first step is a photo. Today, the M-Files MCP Server can serve the *later* steps — find the asset, fetch the procedure — but not the first.

And the market is moving: a construction-drawing vendor (Bluebeam) has already connected its drawing tool to an AI assistant over MCP [20]. "AI + MCP + engineering drawings" is no longer hypothetical in adjacent markets.

The dangerous part: tools that act

Reading tools are low-risk. Tools that **change things** are not. Once an agent can call "create work order" or "move robot arm", every hallucination can become a real-world event.



Three rules of thumb, which Chapter 10 expands:

1. **Least privilege** — an agent gets only the tools and permissions the task needs.
2. **Human approval for writes** — at least until reliability is proven.
3. **Hard limits outside the model** — speed limits, force limits and emergency stops live in the robot's own safety system, never in a prompt.

M-Files' position — "records outcomes and recommends actions; it does not execute transactions" [10] — puts it naturally at the read and record end of this ladder. That's a feature: it's the safest place to be.

Where M-Files stands

The plumbing exists; the eyes don't. M-Files has an MCP Server, an AI Foundation platform planning multiple VMAN agents (including a planned Asset Management / Installed Base Intelligence Agent), and a governed vault behind them. The agent pattern needed for VMAN Vision is well understood and already demonstrated by vendors. What's missing is the first link — a tool that takes a photo and returns grounded, resolvable observations — and the product decision to let MCP clients send images at all.

Chapter 10 — Trust: evaluation, safety and governance

The new hire's first repair

A new technician finishes their first repair and says *"done."* Would you sign it off without looking? Of course not. You'd check: is the leak gone, were the right parts used, was the procedure followed, was the valve isolated first?

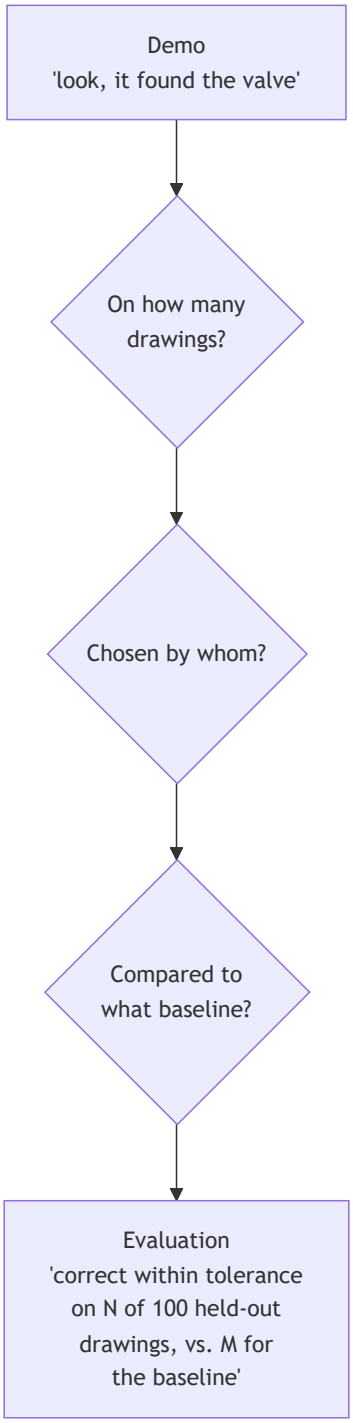
Not because you distrust them personally — because **trust is earned by evidence, and evidence is collected by checking.** AI is no different, except it can be wrong in stranger ways and at far greater speed.

This chapter has three parts: **evaluation** (does it work?), **safety** (what stops it doing harm?), and **governance** (who's allowed, and is it recorded?).

Part 1 — Evaluation: does it actually work?

Demo vs. benchmark

A demo shows that something *can* work. An evaluation shows *how often* it works, on inputs you didn't hand-pick.



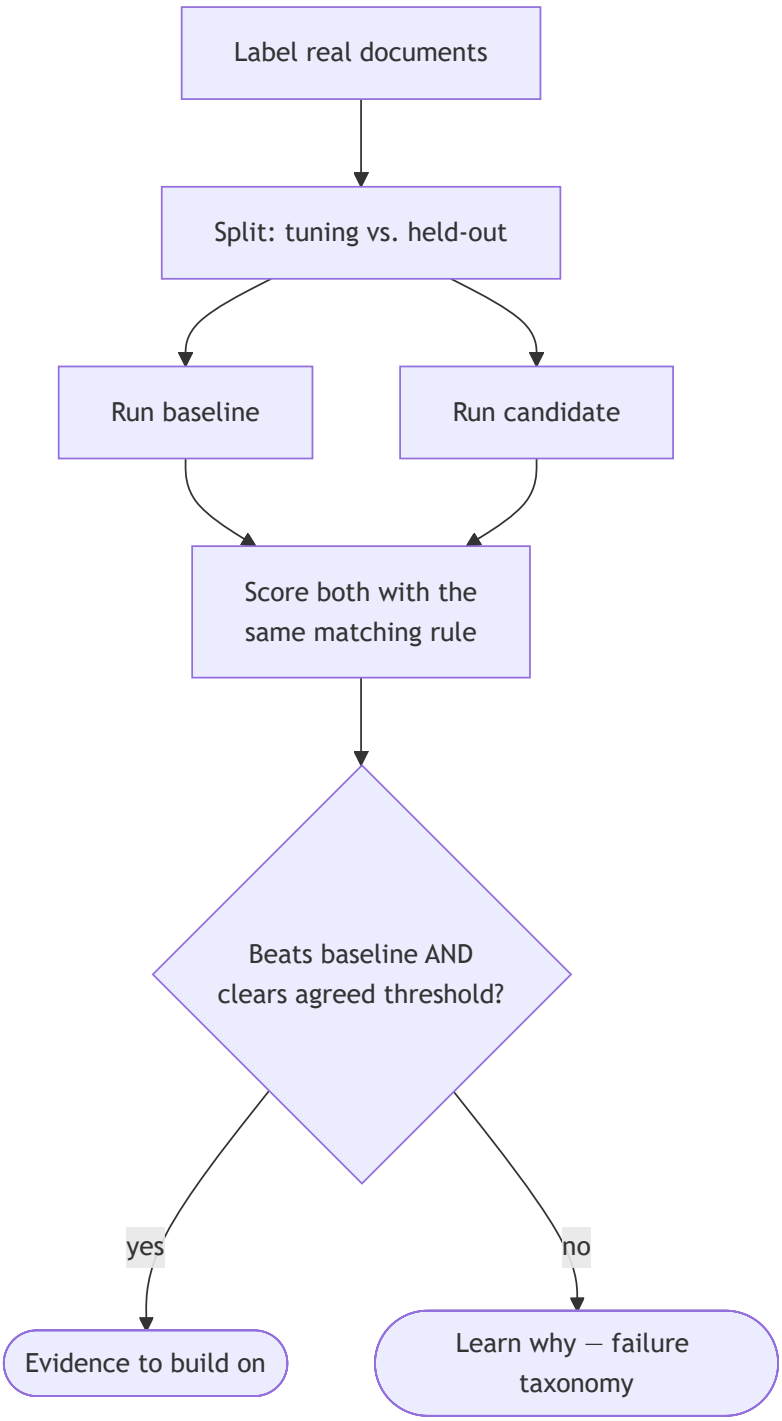
(N and M are placeholders — no such numbers exist yet for this work.)

Everything in this repository today is on the left side of that picture. The repository's own documents are explicit: the comparisons are qualitative, and "point count and label specificity do not measure spatial accuracy."

What a real evaluation needs

To turn a demo into evidence, you need five things:

1. **A labelled set** — a collection of real documents where a human has marked the correct answers (e.g. "the outlet valve is *here*", as a region).
2. **A held-out split** — documents the prompts were never tuned on. Otherwise you're grading your own homework.
3. **A matching rule** — when does a predicted point count as correct? Inside the labelled region? Within 20 units on the 0–1000 grid?
4. **A baseline** — what does the current method (OCR, text-only retrieval) score on the same set?
5. **A threshold agreed in advance** — the repository proposes >90% as a decision criterion, to be set jointly with the teams who'd consume the result, not self-reported.



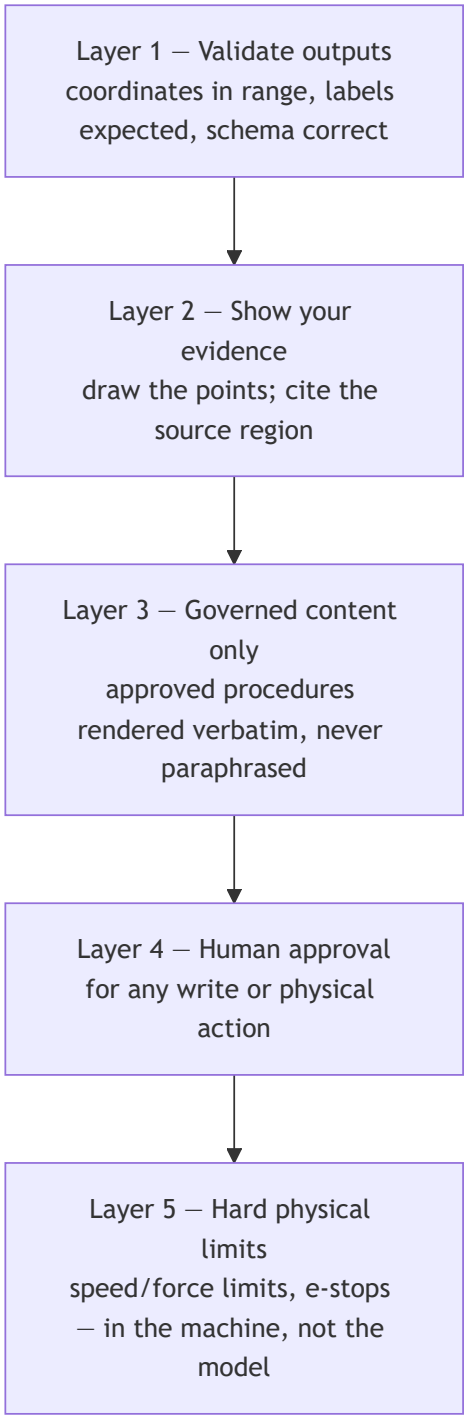
The evaluation questions to always ask

Borrowed from robotics research, and just as useful for document AI:

- **What exactly was tested?** Which documents, which model version, which prompt?
- **What counts as a failure?** Wrong location? Malformed output? Missing item?
- **What changed between runs?** Model versions change silently on preview endpoints.
- **What breaks it?** Rotated scans, handwritten markups, low resolution, unusual symbols.
- **Are failures visible?** In our P&ID example (Chapter 4), a model returned coordinates outside the valid grid — a failure that's invisible unless you validate the output.

Part 2 — Safety: what stops it doing harm?

For document assistance, "harm" mostly means wrong information. For physical agents, it can mean injury. The principle is the same: **layers of protection, with the most important ones outside the AI.**



Two of these layers are unusually natural for M-Files:

- **Layer 3** — the Manufacturing Blueprint's phrase "approved maintenance procedures as governed steps an assistant renders verbatim" [10] is a safety design. The AI finds the right procedure; it

doesn't rewrite it.
- **Record everything** — M-Files "records outcomes" [10]. An audit trail of what the agent saw, what it recommended, and what was done is the foundation of every incident review.

Part 3 — Governance: who's allowed, and where does the data go?

Permissions

A technician on line 3 shouldn't see confidential design files for an unreleased product just because they photographed the machine next to it. Every lookup an agent makes must respect the same permissions the person has. M-Files enforces this in the vault already; any new AI path must go through it, not around it.

Where the data goes

This is the decision that will come up in every stakeholder conversation:

Option	Where images go	Governance	Grounding quality (in our runs)
Azure-native (Foundry, Document Intelligence, Content Understanding)	Stays inside the customer's Microsoft tenant boundary	Easiest to approve; aligned with our strategy [9]	Good for text/layout; weaker at precise pointing
Gemini Robotics-ER 2 via Google API	Sent to Google	Needs an approved external dependency	Strongest pointing we've observed — preview model
Gemini as offline benchmark only	Only approved test data goes to Google	Low risk	Used to *measure* Azure options, not in production
Microsoft Magma-8B on Foundry Managed Compute (preview)	Stays in our Azure subscription, on dedicated GPUs	Only if available in our project/region; needs a GPU quota request; model card excludes production use	Untested here; strongest published icon grounding in its paper
Self-hosted open model (e.g. Magma weights on our own GPU)	Your own machine	Full control	Untested here; operational cost

The repository's recommended stance is **start Azure-native**, and use Gemini-style grounding as a benchmark or add-on for the visual cases Azure can't yet solve [18].

Vendor-specific rules worth knowing

From Google's documentation, checked 2026-09-18 [17]:

- **Unrestricted API keys are rejected** — keys must be restricted in AI Studio. Never commit a key to Git (this repository's `.env` is gitignored for exactly this reason).

- **The Interactions API stores requests by default.** Use `store=false` where data-handling policy requires stateless requests.
- **Video and audio with identifiable people** require notice, consent and minimisation. Don't use people-containing media in any experiment without an approved privacy plan.
- **ER 2 is a preview endpoint** — output format and behaviour can change without notice. Pin versions and re-run evaluations when they change.

Customer data in repositories

This repository learned this the practical way: customer drawings were removed from the demo in favour of public, licence-tracked fixtures (`fixtures/public-fixtures.json`) — but files that were committed earlier still exist in the private Git history. Treat that as a standing rule: **never commit customer content to a research repository in the first place**, because removing it later doesn't erase it.

> ### Where M-Files stands
>
> **Governance: strong. Evaluation: missing.** M-Files' permissions, verbatim governed content, and audit trail are exactly the safety properties physical agents need — a genuine competitive advantage. But there is no labelled evaluation set, no scoring rubric, and no measured accuracy anywhere in this work yet. Until there is, every claim must be phrased as "demonstrated on these examples", never "works". Building a small, labelled, held-out evaluation set is the highest-leverage next step after the first demo — and it's what turns a prototype into something a product owner can commit to.

Chapter 11 — Putting it together: VMAN Vision

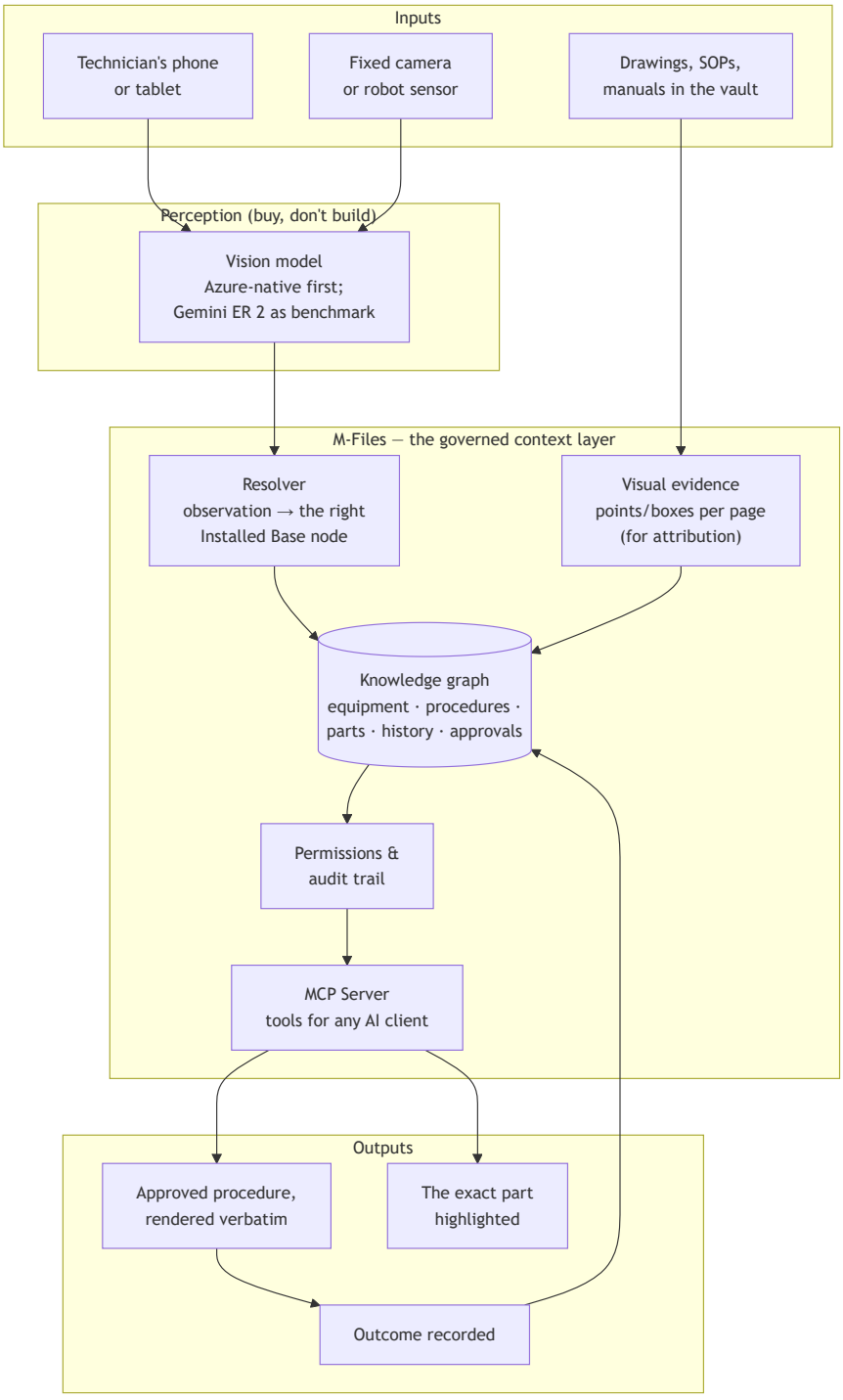
Assembling the pieces

Every chapter so far has handed you one piece:

Piece	Chapter	M-Files today
Eyes — see the scene	3	Missing (buyable)
Finger — point at the exact thing	4	Prototype only
Memory — know which thing it is	7	Strong shape; key module planned
Search — find the right content	8	Text-only
Hands — call the right systems	9	MCP exists; no image input
Judgement — check it's right and allowed	10	Governance strong; evaluation missing

This chapter assembles them into one architecture, places it in the market, and lays out how to get there step by step.

The architecture

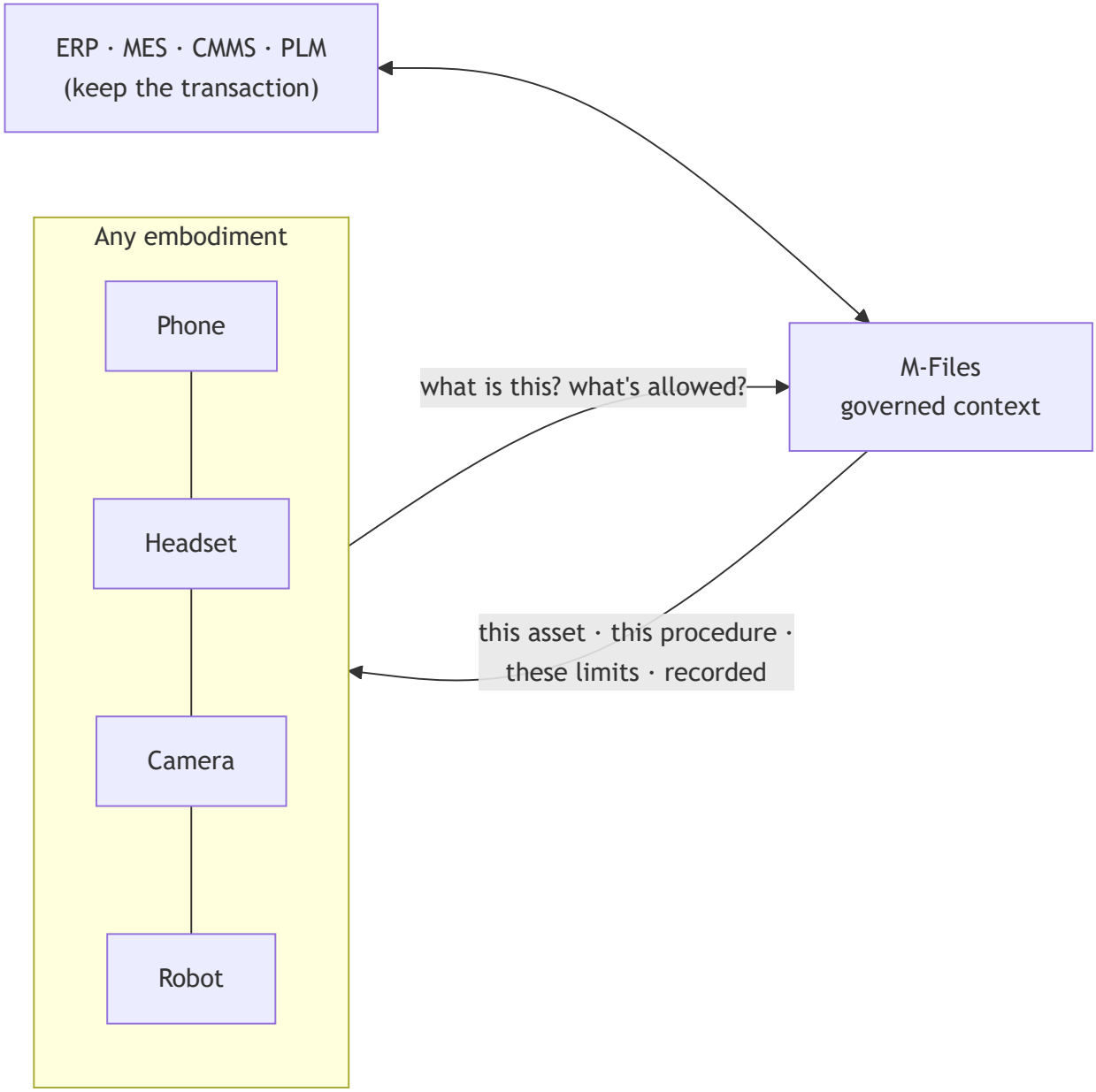


Three design choices are baked into that picture:

1. **Perception is bought, not built.** The vision model is a replaceable component behind an interface — which is exactly how `prototype/providers/` is already structured.
2. **M-Files owns resolution and governance.** The step from "a pump" to "*this* pump, *its* approved procedure, *your* permission to see it" is the defensible part.
3. **M-Files never executes.** It recommends, renders, and records. The customer's own systems — or a partner's robot and its safety systems — do the transaction [10].

Physical Agents as a Service

Stretch that architecture forward and you get the long-term idea from the pitch: **Physical Agents as a Service (PAAS)**. Any embodiment — a phone, a headset, a fixed camera, a robot arm, a humanoid — asks M-Files the same question: "*what is this, what's its state, and what's the approved way to act on it?*"



"We don't build robots. We become the layer every robot needs before an enterprise will trust it with a task." That sentence is consistent with the Blueprint's non-compete stance — the Blueprint's own line, "your systems keep the transaction, M-Files keeps the governed context" [10], is PAAS in one clause.

The competitive landscape

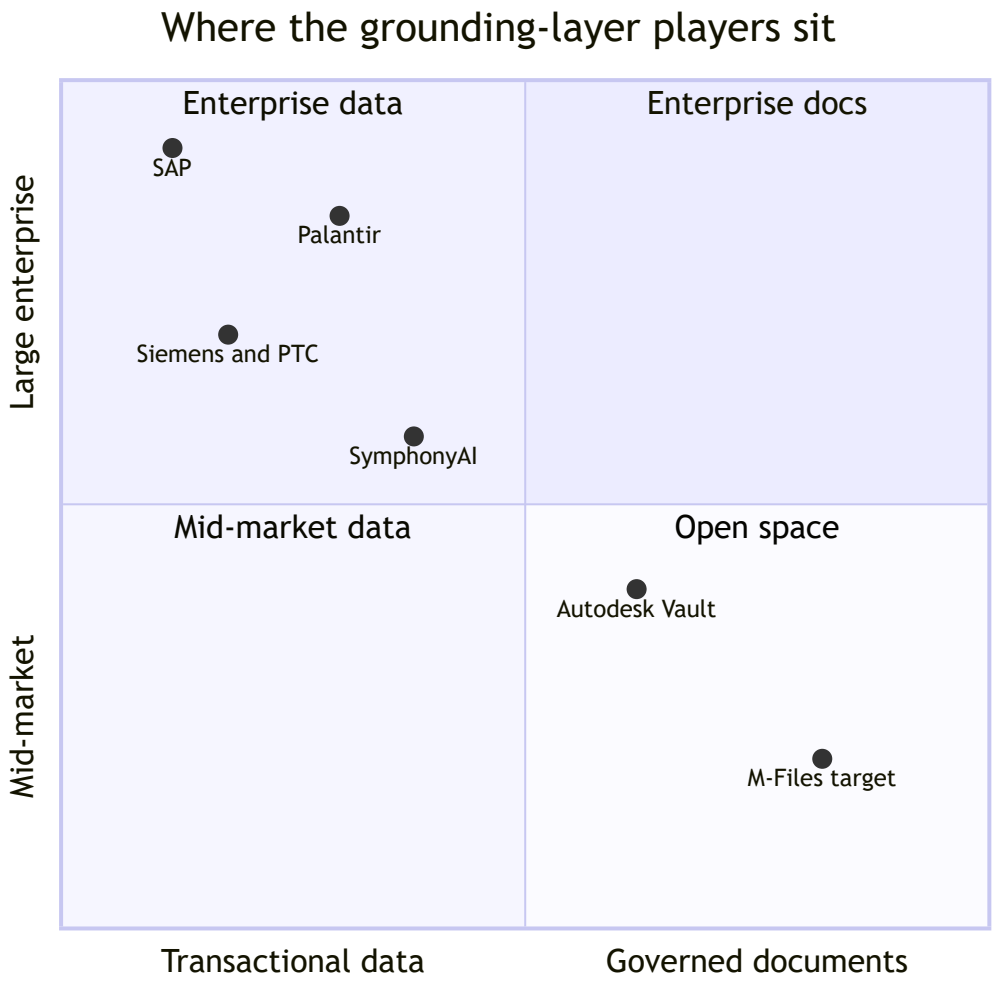
Honesty first: M-Files is **not** early to the idea of grounding physical AI in enterprise data.

Player	What they're doing	Grounded in	Threat or partner?
SAP	Knowledge Graph + embodied-AI pilots (Tetra Pak lab PoC with a humanoid running warehouse workflows) [13]	ERP transactions	Partner-shaped: they keep the transaction
Palantir	Ontology of factory → line → machine → part with telemetry [14]	Sensor/operational data	Competes for "the context layer" at large enterprises
Siemens, PTC, Dassault	Digital twins wired to NVIDIA simulation via OpenUSD [15]	CAD geometry & simulation	Partner-shaped: CAD/PDM we "wrap around, not replace" [12]
SymphonyAI IRIS Foundry	VLM P&ID extraction matched to CMMS/EAM asset graphs	Industrial asset systems	Closest direct analog — for asset systems, not a document vault
Autodesk Vault 2027 Assistant	Conversational AI over vault metadata	Existing vault metadata (not pixels)	Raises the bar: "chat with your vault" alone is no longer differentiated
Drawing-AI point solutions (DiagramIQ, Pathnovo, LlamaParse…)	VLM-based drawing digitisation	The drawing itself	Reading drawings with a VLM is no longer novel

Sources for the last three rows: `pitch/README.md` "Pitch-relevant updates" and `RESEARCH_LOG.md`, 2026-08-23 to 2026-08-26.

So where's the open space? Every player above grounds agents in *structured transactional data* or in *the drawing alone*. No one found so far does it for a **governed, general-purpose document and records vault** — the manuals, SOPs, marked-up drawings, bulletins and approvals where much of a plant's real

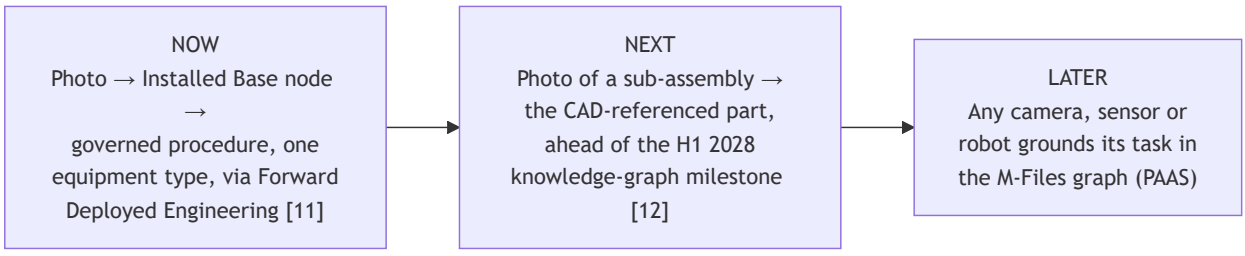
operational knowledge lives. And the CAD Integration Roadmap already points M-Files at **SMB and mid-market first** [12] — customers who will never run a Palantir-scale ontology programme.



(Positions are qualitative judgements for discussion, not measurements.)

Start small, land big

The McKinsey reliability ladder from Chapter 6 — demo → controlled reliability → production dependability → unit economics — maps directly onto a sequence of steps, each one tied to something already on M-Files' own roadmap:



Step	What gets built	Depends on	McKinsey rung
Now	One equipment type, one pilot customer: photo in, correct asset + verbatim procedure out, plus a labelled evaluation set	Blueprint entity model [10]; Forward Deployed Engineering route [11]	1 → 2
Next	Extend resolution to CAD/PDM-linked components; boxes and crop-and-re-inspect	CAD Integration Roadmap [12]	2 → 3
Later	Embodiment-agnostic API; first robotics or platform partner	Proven resolution + partner	3 → 4

Each step is small enough to be approved on its own and produces evidence the next step needs.

Questions a product owner will ask (and honest answers)

Question	Honest answer
"Are we building a robotics product?"	No. M-Files recommends and records; it never executes [10]. The embodiment is the customer's or a partner's.
"Doesn't Document Intelligence already do this?"	It extracts text and layout with reliable citations, and should be the foundation. It doesn't locate symbols or answer visual questions. Complementary, not duplicate (Chapters 4, 8).
"Why not just use Gemini?"	Strongest pointing we've observed, but a preview endpoint outside Microsoft's boundary. Start Azure-native; use Gemini as a benchmark (Chapter 10).
"How accurate is it?"	Unknown. Everything so far is qualitative. The first deliverable should include a labelled evaluation set (Chapter 10).
"Isn't the Installed Base module 2027–28 work?"	It's validated design, not yet built [11]. A working photo-to-node demo is a concrete reason to pull part of it forward.
"Who else is doing this?"	SAP, Palantir, Siemens and others for structured data; point solutions for drawings. Nobody found yet for a governed document vault (see table above).

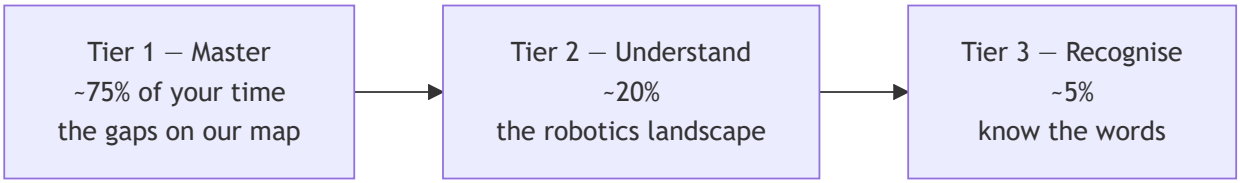
> ### Where M-Files stands
>
> **A credible, differentiated position that is still mostly on paper.** The architecture uses M-Files' real strengths (the graph, governance, the partner-shaped non-compete stance) and buys what it lacks (perception). The competitive gap — grounding physical agents in governed *documents* rather than transactions — is real but will not stay open forever. The path from here is deliberately small: one equipment type, one pilot, one labelled evaluation set, delivered through a route M-Files already sanctions.

Chapter 12 — The hero's path: a curriculum

Learning to swim

Nobody learns to swim by reading about fluid dynamics. You get in the shallow end, you splash, you learn one stroke, then another, and eventually the deep end stops being scary.

This chapter is the shallow-to-deep path. It's re-weighted from the earlier six-page personal guide this book grew out of, which was a general robotics curriculum (control theory, reinforcement learning, simulators). Chapter 1's map showed that M-Files' real gaps are **seeing, pointing, retrieving, remembering and proving** — so that's where most of the time goes now.



Tier 1 — Master these

1.1 Vision and VLMs (Chapter 3)

- **Read:** *Attention Is All You Need* (arXiv 1706.03762), *Vision Transformer* (arXiv 2010.11929), *CLIP* (arXiv 2103.00020).
- **Course:** Stanford CS231n (vision) — the lectures on CNNs, transformers and vision-language models.
- **You're done when:** you can explain to a colleague why a VLM can describe a P&ID but OCR can't — and name three ways VLMs fail.

1.2 Spatial grounding (Chapter 4)

- **Read:** Google's Gemini Robotics overview and the ER 2 quickstart notebook; this repository's `docs/gemini-robotics-model-guide.md`.
- **Read:** *Magma* (arXiv 2502.13130) — Microsoft's mark-based alternative — then this repository's `docs/magma-readiness.md`.
- **Do:** run the demo (Chapter 13), then reproduce the P&ID comparison from Chapter 4.
- **You're done when:** you can convert `[y, x]` on the 0–1000 grid to pixels in your head, and explain point vs. box vs. crop-and-re-inspect.

1.3 Retrieval and RAG (Chapter 8)

- **Read:** the original *RAG* paper (arXiv 2005.11401); Microsoft's *GraphRAG* (arXiv 2404.16130); Azure's multimodal search overview.
- **Also excellent:** the RAG and Context Engineering chapters of *AI Engineering: System Design Patterns for LLMs, RAG and Agents* (Daily Dose of Data Science, 2025) — the book whose style this one borrows.
- **You're done when:** you can draw classic RAG, GraphRAG and agentic RAG from memory and say which fits which M-Files question.

1.4 Knowledge graphs (Chapter 7)

- **Read:** this repository's Chapter 7, then the Manufacturing Blueprint [10] and Product Strategy 2025–26 [9] on Confluence.
- **You're done when:** you can sketch the Installed Base & Maintenance Knowledge graph and explain the difference between perception and resolution.

1.5 Agents, tools and MCP (Chapter 9)

- **Read:** *ReAct* (arXiv 2210.03629); the Model Context Protocol documentation; the Agents and MCP chapters of *AI Engineering* (above).
- **You're done when:** you can write a tool description, trace an agent loop step by step, and explain API vs. function calling vs. MCP.

1.6 Evaluation and trust (Chapter 10)

- **Read:** the evaluation plan in the sibling repository (`complex-document-grounding-poc/docs/evaluation-plan.md`) and this repository's comparison docs, paying attention to how they phrase limitations.
- **You're done when:** you can design a labelled evaluation for point grounding — matching rule, held-out split, baseline, threshold — on one page.

Tier 2 — Understand these

2.1 World models (Chapter 5)

- **Read, in order:** *World Models* (arXiv 1803.10122), *DreamerV3* (arXiv 2301.04104).
- **Watch:** Jim Fan, "Robotics' End Game" (Sequoia AI Ascent 2026) [1].
- **You're done when:** you can explain encoder → latent state → dynamics → decoder, and why an LLM isn't automatically a world model.

2.2 VLAs and robot learning (Chapter 6)

- **Read:** *RT-2* (arXiv 2307.15818), *OpenVLA* (arXiv 2406.09246), *Diffusion Policy* (arXiv 2303.04137).

- **Watch:** McKinsey Live, "Physical AI's Hidden Value Pools Leaders Miss" (Sept 2026) [5] — for the data pyramid and reliability ladder.
- **You're done when:** you can explain action chunks, diffusion policies and the data pyramid to a product manager.

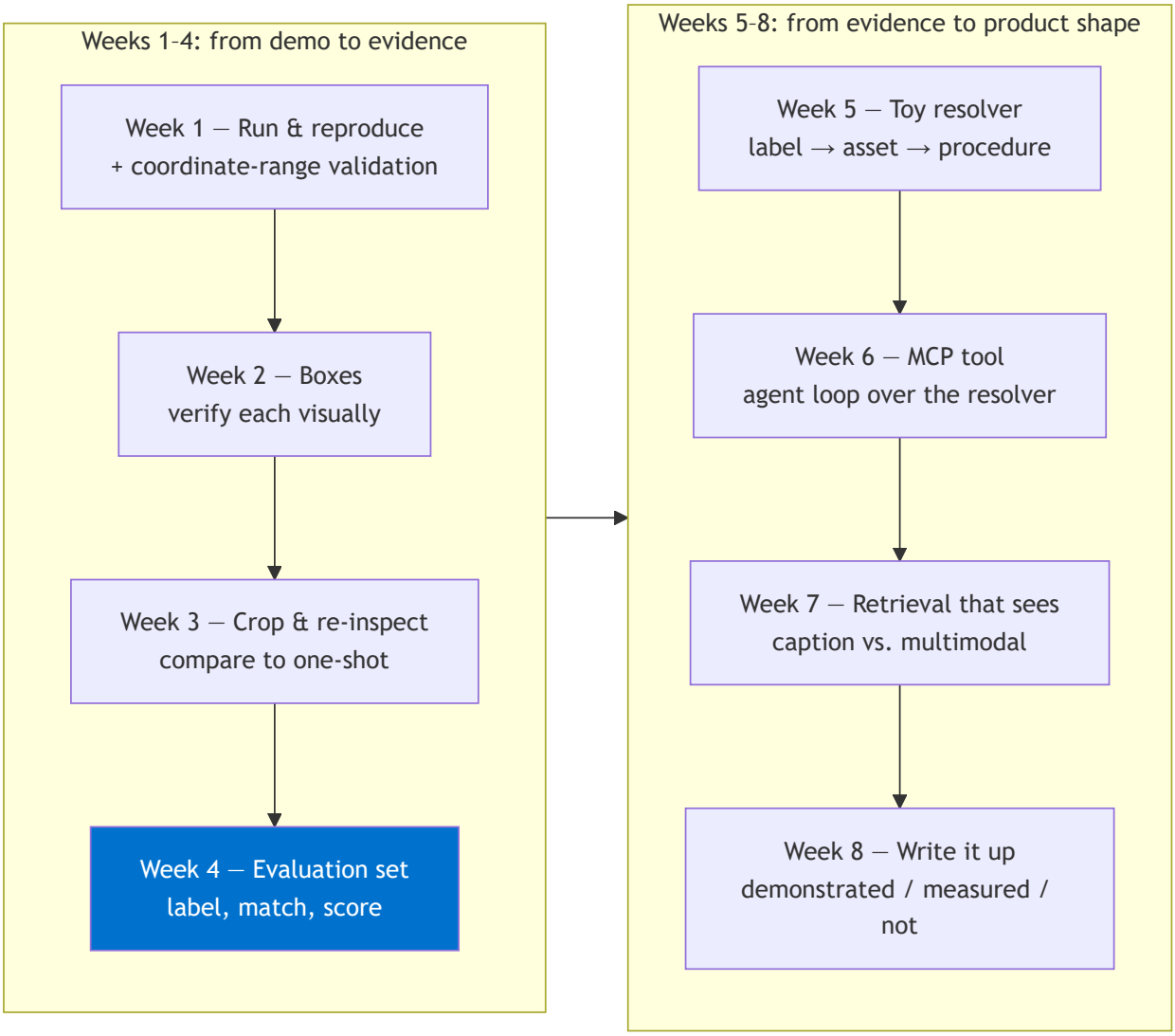
Tier 3 — Recognise these

You don't need to study these for VMAN Vision. You need to recognise the words when a robotics partner uses them.

Topic	One-line meaning	If you ever go deeper
PID / LQR / MPC	Classic ways to control a machine from its sensors	MIT Underactuated Robotics
Kalman filter	Estimating true state from noisy sensors	Same as above
Kinematics / inverse kinematics	Joint angles ↔ where the hand ends up	*Modern Robotics* (Lynch & Park)
PPO / SAC	Popular reinforcement-learning algorithms	Berkeley CS285
Sim-to-real	Transferring a policy from simulation to hardware	NVIDIA Isaac Lab
MuJoCo	A standard physics simulator for research	mujoco.org

The capstone ladder (8 weeks, M-Files-shaped)

Each capstone produces **evidence**, following this repository's conventions (see `AGENTS.md` and the `physical-agent-experiment` skill — load it before starting any new experiment). Use public, licence-tracked fixtures only.



Week	Build	Evidence to keep
1	Reproduce the demo; reject coordinates outside 0–1000	Test for the validation; before/after overlays
2	Box output on public fixtures	Raw responses, overlays, visual verification notes
3	Crop and re-inspect	One-shot vs. crop answers side by side
4	Labelled evaluation set + scoring script	Labels, matching rule, per-model scores, failure list
5	Toy Installed Base resolver	The mini graph, resolver tests, example runs
6	Resolver exposed as an MCP tool	Tool schema, agent-loop transcript

Week	Build	Evidence to keep
7	Caption vs. multimodal retrieval	Queries, results, which one found the right figure
8	Evidence write-up	A doc in `docs/`, labelled demonstrated / measured / untested

Week 4 is the most important week on this list. Everything after it can be measured; everything before it is a demo.

Final advice

The shortest route to real competence is not chasing the newest model name. It's learning to say, precisely:

- **What** the model was asked,
- **Where** its answer points,
- **Which** object in our graph that resolves to,
- **How often** it's right, measured, and
- **Who** is allowed to see and act on it.

Get those five right and the model becomes a replaceable part. That's exactly the position M-Files wants to be in.

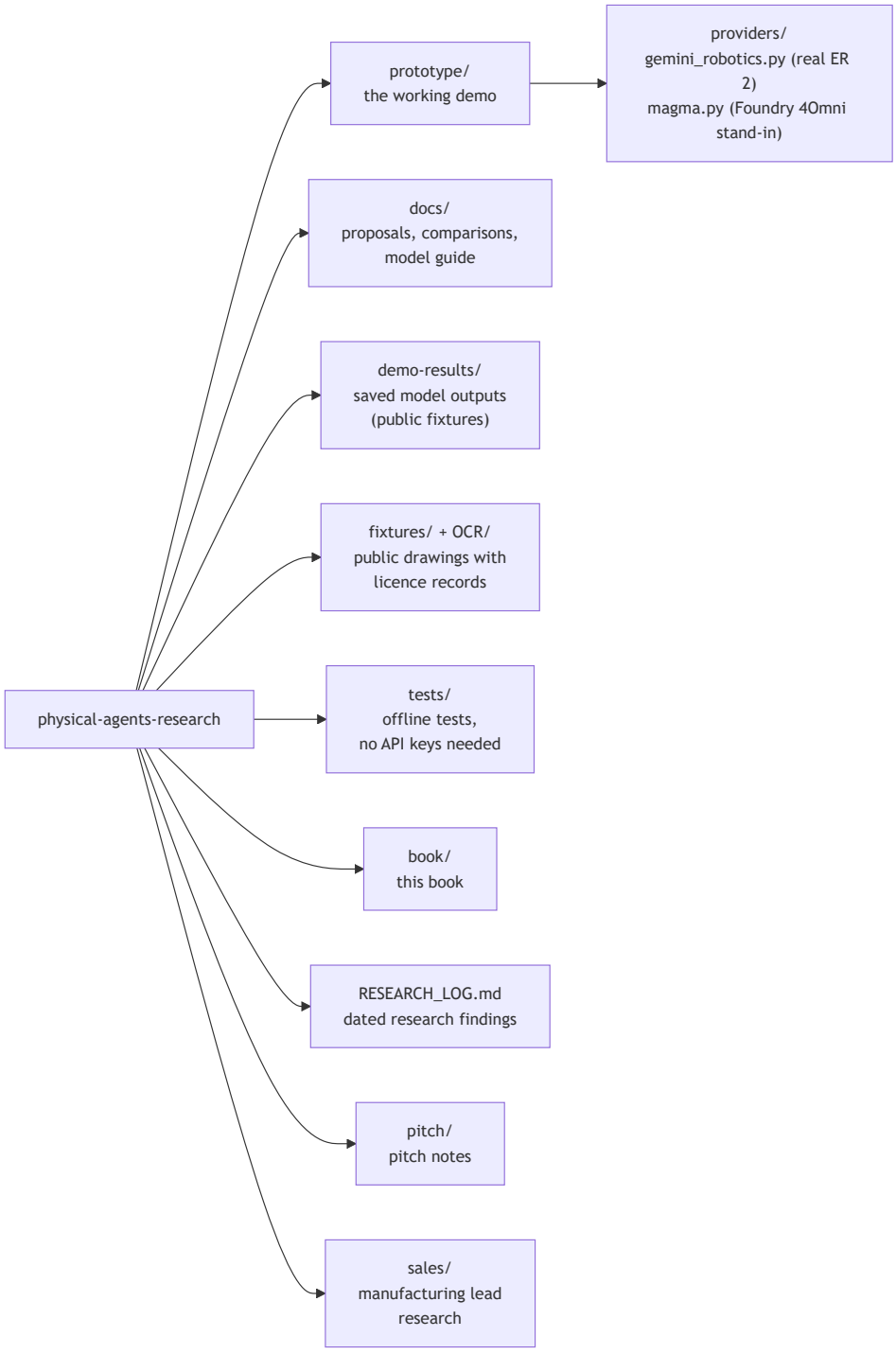
Chapter 13 — The handover: your first week

Written 28 September 2026 for Ankit, taking over from Viditya while he's away (1–6 October). Dates and statuses below are as of that day — check `git log` and the PR list for anything newer.

The relay race

In a relay, the baton handover is where races are won and lost. The incoming runner starts moving *before* the baton arrives, so there's no standing start. This chapter is designed to give you that running start: where everything is, what's proven, what's open, and who's who.

The repository in one picture



Path	What it is	Read it when
`AGENTS.md`	The rules for every contributor and coding agent	**First, before any change**
`prototype/`	Flask web demo + CLI; providers isolated under `providers/`	You want to run or change the demo

Path	What it is	Read it when
`docs/gemini-robotics-model-guide.md`	What Google's models can and can't do, with sources	Before any model conversation
`docs/complex-document-retrieval-visual-grounding-proposal.md`	The "visual evidence layer" proposal and experiment design	Before talking to retrieval/MCP teams
`docs/differentiation-plan-vs-metadata-poc.md`	Where we do and don't overlap with the AI-Metadata team	Before proposing anything to them
`docs/*-comparison.md`	Gemini vs. Foundry qualitative comparisons	When someone asks "which model?"
`RESEARCH_LOG.md`	Dated findings from the research routine	To catch up on the market
`pitch/README.md`	Pitch-relevant updates, including competitor corrections	Before any stakeholder pitch

---

Running the demo (about 15 minutes)

```
cp .env.example .env
```

Fill in `GEMINI_API_KEY` (and `FOUNDRY_API_KEY`, `FOUNDRY_ENDPOINT`, `FOUNDRY_DEPLOYMENT` for the Azure provider). Ask Viditya for access rather than creating new keys; never commit `.env`.

```
pip install -r prototype/requirements.txt
```

```
python -m prototype.server
```

Open `http://127.0.0.1:5000`, pick a public sample, choose a provider, run it. Points are drawn back onto the drawing next to the raw JSON.

Two things to know:

- **Results are cached** in `.vman-vision-cache/` (gitignored) — re-running the same file + provider + prompt costs no tokens.
- **The latest result per document is written to `demo-results/`**, which *is* tracked. Before committing, check nothing from a non-public upload landed there.

Run the offline tests (no keys needed):

```
python -m pytest tests -q
```

The rules that matter most

From `AGENTS.md`, in order of how often they'll matter:

1. **Topic branch → pull request → Viditya (or the owner) merges.** Never push code to `main`. No auto-merge.
2. **No customer content, credentials, raw provider output or unapproved overlays in Git.** Public, licence-tracked fixtures only. (Customer files removed from the working tree earlier still exist in private history — don't add more.)
3. **Treat model output as experimental evidence.** Report failure modes; never invent confidence values.
4. **Label every claim:** demonstrated, documented, or hypothesis (see the book's README).
5. **Scope:** complex-document comparisons (Document Intelligence, Content Understanding, metadata integration) live in `complex-document-grounding-poc`, not here.

The repository also defines three agent roles (`.github/agents/`, mirrored in `.claude/agents/`): `research-producer`, `physical-agent-developer`, `experiment-qa`, and a `physical-agent-experiment` skill to load before any new experiment.

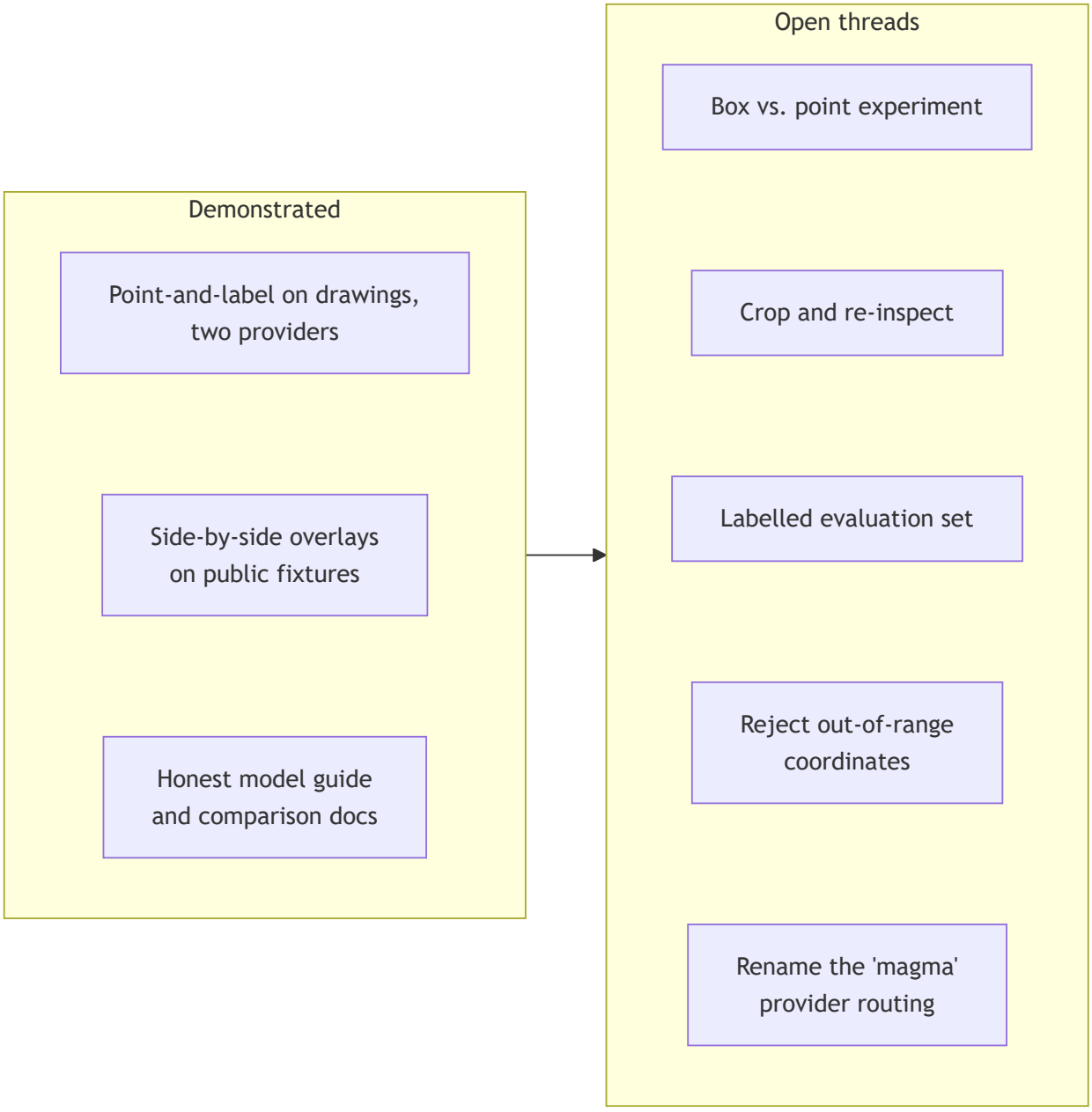
What runs on its own

Three scheduled cloud routines run under Viditya's account and write to this repository:

Routine	When (UTC)	What it does
Biweekly Physical Agents Research	18:30 on the 1st and 15th of each month	Appends a dated entry to `RESEARCH_LOG.md`; pushes to `main` (allowed exception)
Weekly Physical Agents Repo Update	Sundays 19:00	Synthesises the week's log; opens a PR only if something justifies a change
Weekly Manufacturing Win Strategist	Sundays 20:00	Updates `sales/manufacturing/` lead profiles

The research routine moved from nightly to **biweekly on 18 September**, so there were no research entries between 17 September and 1 October — that's expected. The next one runs **1 October**. (The 27 September weekly synthesis flagged the silence as a possible fault; it isn't.)

What's proven, and what's open



Open thread	Why it matters	Where to start
Labelled evaluation set	Nothing is measurable without it (Chapter 10)	Capstone week 4 (Chapter 12)
Box vs. point	The next rung of grounding (Chapter 4)	The proposal doc's "Scoped follow-up" section
Crop and re-inspect	Fixes small-detail failures	Same section
Out-of-range coordinates	Foundry returned x = 1050 / 1100 on the P&ID fixture; the overlay silently clamps them (Chapter 4)	`prototype/overlay.py` `_to_pixels`, plus a test

Open thread	Why it matters	Where to start
`magma` naming	The provider is Azure Foundry `4Omni`, not Microsoft Magma — the name misleads	`prototype/providers/magma.py` and demo routing
Resolver prototype	The core M-Files differentiator (Chapter 7) has no code yet	Capstone week 5
First real Magma run	Magma-8B is listed in the Foundry catalog. It may be deployable inside Azure on Foundry Managed Compute (preview), depending on project, region and quota. It's the Microsoft-native grounding candidate	First check **Available in my Project** in Foundry; then `docs/magma-readiness.md` §9–10; rename the `magma` provider first

---

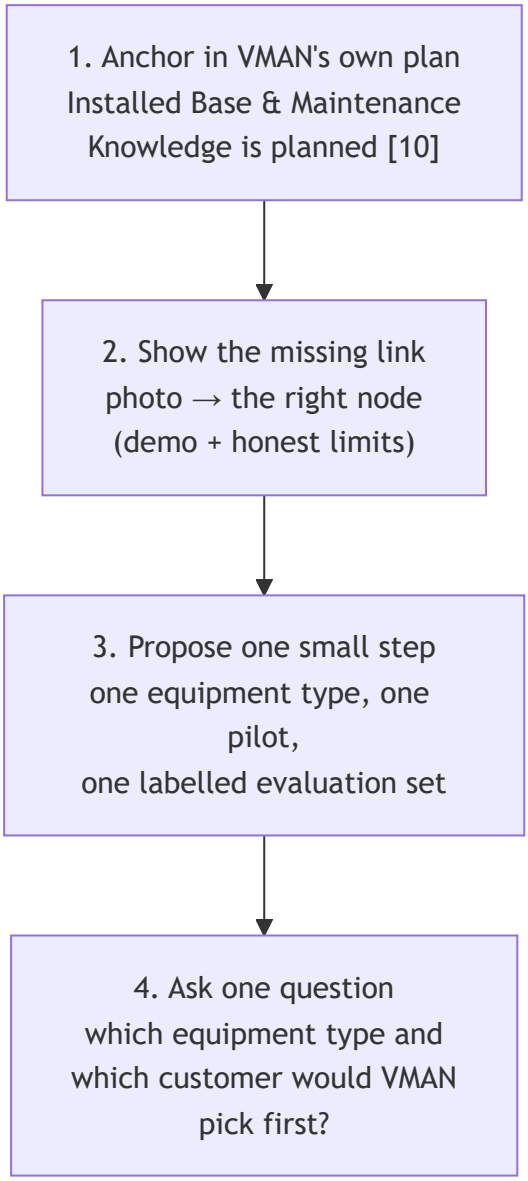
Who's who

Person / team	Role in this story
Rajat Singh	VMAN Product Owner
Matti Kurittu	VMAN Team Architect
Ville Somppi	Owner of the Manufacturing Blueprint page [10]
Sivaram Reddy	Author of the CAD Integration Roadmap 2026–2029 [12]
Tony Grout	Author of the Product Strategy 2025–26 blog [9]
Vatsal Kariya	MCP Server; owns the beta page with the image-reading limitation
AI-Metadata team	Owns metadata extraction (`metadata-poc`, parsing engine)
Q4 "Complex Document Retrieval and Foundation" group	Vatsal Kariya, Antti Nivala, Raphaël Allègre, Brian Remmington — aligning MCP, metadata-agent and platform on complex-document retrieval

---

Preparing for a conversation with the VMAN Product Owner

A conversation with Rajat Singh is planned for 7 October. You don't need to pitch the whole vision — you need to connect it to what VMAN already plans.



Five sentences to have ready:

1. "*We're not proposing a new product — we're proposing the perception link for the Installed Base module your Blueprint already plans.*"
2. "*M-Files keeps the governed context and records outcomes; it never executes — that's straight from the Blueprint.*"
3. "*The demo shows it can point at the right thing on real engineering drawings. We haven't measured accuracy yet; the first deliverable would include a labelled evaluation set.*"
4. "*Azure-native first. Gemini is our benchmark, not a dependency.*"
5. "*Forward Deployed Engineering is the route — one real customer, one equipment type.*"

Viditya holds the pitch deck and the one-page summary outside this repository; ask him for the latest versions before the meeting.

Keeping this book current

The book is published in numbered volumes. You're reading Volume 2, and every past volume's PDF is kept in `book/volumes/`. When you change something substantive — a new section, or a fact that changes a conclusion — cut a new volume:

1. Bump `book/volume.json`.
2. Update the volume line in `book/README.md`.
3. Add a row to VOLUMES.md.
4. Run `python book/tools/build_pdf.py --release`.

Typo fixes don't need a new volume.

If you get stuck

- **A claim you're unsure of** — check Appendix B for its source and verification date. If it's older than a month, re-verify before repeating it.
- **A model behaves oddly** — ER 2 is a preview endpoint; check whether the model version changed before debugging your code.
- **A scope question** — `AGENTS.md` "Purpose and boundaries" first; then ask Viditya on his return.

Welcome aboard.

Appendix A — Glossary

Term	Plain-language meaning	Chapter
Action chunk	A short sequence of future robot actions predicted in one go, instead of one command at a time	6
Agent	A system where a model repeatedly decides what to do next — call a tool, observe, decide again — until the goal is met	9
Agentic RAG	RAG where an agent decides what to search, when, and whether to search again	8
Attribution (source)	Pointing back to where an answer came from — document, page, and ideally region	4, 8
Box (bounding box)	A rectangle around an object; in Gemini ER 2, `[ymin, xmin, ymax, xmax]` on a 0–1000 grid	4
CLIP	A 2021 model that put images and text into one shared "meaning space"	3
CNN	Convolutional neural network — slides small learned filters over an image to find edges, shapes, objects	3
Compounding error	Small prediction mistakes that snowball when a model predicts many steps ahead	5
Content Understanding (Azure)	Azure service that extracts content with location metadata and can describe figures	8, 10
Crop and re-inspect	Zoom into a located region and ask a second, focused question	4
Data pyramid / ladder	Cheap video at the base, simulation in the middle, scarce real robot data at the top	6
Decoder	Turns a compact internal representation back into an output (image, text, action)	5
Diffusion policy	A robot policy that generates a distribution of action sequences and picks one coherent option	6
Document Intelligence (Azure)	Azure's OCR and layout service — text, tables, regions, with reliable citations	3, 8
Dynamics	The part of a world model that predicts the next state given the current state and an action	5
Embedding	A list of numbers representing the meaning of text or an image, so similar things sit close together	3, 8
Encoder	Compresses a large input (an image) into a compact representation (latent state)	5
Evaluation set (labelled)	Real inputs with human-marked correct answers, used to measure accuracy	10

Term	Plain-language meaning	Chapter
Forward Deployed Engineering	M-Files' route for proving ideas with a real customer ahead of the core roadmap [11]	1, 11
Function calling / tool calling	A model requesting that your system run a described function	9
GraphRAG	RAG that follows knowledge-graph relationships, not just nearest-text matches	8
Grounding (spatial)	Connecting words to locations — "where exactly is it?" as coordinates	4
Hallucination	A model confidently producing something untrue — in vision, including points on things that aren't there	3, 10
Held-out set	Evaluation data the prompts and models were never tuned on	10
Installed Base & Maintenance Knowledge	The planned M-Files module modelling equipment by serial number with procedures, bulletins, parts, suppliers and warranty [10]	7
Interactions API	Google's newer unified API for Gemini; stores requests by default unless `store=false`	10
Knowledge graph	Things (nodes) and named relationships between them (edges), queryable and explainable	7
Latent state	A model's compact internal summary of "what's going on"	5
MCP (Model Context Protocol)	An open standard for exposing tools, resources and prompts to any AI application	9
MCP client / server	The AI application that uses tools / the system that exposes them	9
Model-based vs. model-free	Learning how the world works then planning, vs. learning actions directly by trial and error	5
Normalised coordinates	Positions expressed as fractions of the image (0–1000), independent of image size	4
OCR	Optical character recognition — reading the characters on a page	3
P&ID	Piping and instrumentation diagram — the standard drawing of a process plant's pipes, valves and instruments	3, 4
PAAS (Physical Agents as a Service)	The long-term idea: any embodiment asks M-Files "what is this and what's allowed?"	11
Perception vs. resolution	Turning pixels into "a pump" vs. turning "a pump" into *this specific pump* in the graph	7
Physical agent	A system whose decisions work through sensors, bodies or tools in a real, changing environment	2
Point	A single `[y, x]` location on a 0–1000 grid with a label	4
RAG	Retrieval-augmented generation — retrieve your own content, then have a model answer from it	8
ReAct	The reason → act → observe loop behind most agents	9

Term	Plain-language meaning	Chapter
Reinforcement learning (RL)	Learning actions from reward by trial and error	6
Set-of-Mark (SoM)	Overlay numbered marks on candidate regions of an image so the model answers by choosing a mark; Magma's grounding method [21]	4
Sim-to-real	Transferring a policy learned in simulation onto real hardware	6
Tool	A function or system action an agent is allowed to request	9
Trace-of-Mark (ToM)	Tracking marks across video frames so a model learns to predict where things will move next; Magma's planning method [21]	4
Transformer	The attention-based architecture behind modern language and vision models	3
VLA	Vision-language-action model — camera + instruction in, robot actions out	6
VLM	Vision-language model — sees images and talks about them	3
ViT (Vision Transformer)	A transformer that treats image patches like words	3
World model	A model that predicts how the world's state changes given an action	5

Appendix B — Sources and verification status

Every bracketed number in this book points here. Numbers [1]–[12] match the VMAN Vision pitch deck's Sources slide so the two stay consistent.

How to read the "Verified" column: the date the claim was last checked against the source itself. Anything older than about a month should be re-checked before it's repeated to a stakeholder — especially preview model capabilities and competitor moves.

External sources

#	Source	Used for	Verified
[1]	Jim Fan (NVIDIA), "Robotics' End Game," Sequoia Capital AI Ascent 2026 — youtube.com/watch?v=3Y8aq_ofEVs	Robotics repeating the LLM playbook; data ladder; world action models	2026-09 (transcript)
[2]	Starship Technologies / S-Group, 1M+ grocery deliveries in Finland — starship.xyz	Pitch deck context	2026-09
[3]	Wolt + Coco robot delivery launch, Turku, Finland — cocodelivery.com	Pitch deck context	2026-09
[4]	Waymo scaling targets — forbes.com, techcrunch.com	Pitch deck context	2026-09
[5]	McKinsey Live, "Physical AI's Hidden Value Pools Leaders Miss" (Ani Kelkar & Mark Patel), Sept 2026 — youtube.com/watch?v=Lt7K3jdBsNo	$370B 2040 market; data pyramid; reliability ladder	2026-09 (transcript + frames)
[6]	This repository — github.com/M-Files-AI/physical-agents-research	All "demonstrated" claims	2026-09-28
[7]	Google DeepMind, Gemini Robotics — deepmind.google/models/gemini-robotics	Model family overview	2026-09-18
[8]	LinkedIn Learning, "Physical AI Architecture Foundations: Designing Autonomous Machines with Agentic AI"	Background reading (pitch deck)	—
[13]	SAP News, physical-AI robotics pilots; SAP Sapphire 2026 announcements — news.sap.com	SAP Knowledge Graph; Tetra Pak / RobotEra lab PoC	2026-09-25 (web search)
[14]	Palantir Ontology / AIP — palantir.com/platforms/ontology	Factory → line → machine → part ontology	2026-09-25 (web search)

#	Source	Used for	Verified
[15]	Siemens CES 2026 industrial-AI announcements; NVIDIA physical-AI ecosystem announcements; PTC Onshape → Isaac Sim	Digital twins, OpenUSD, partner ecosystem	2026-09-25 (web search)
[16]	Industry reporting on factory humanoid deployments (Schaeffler/Humanoid, BMW, Samsung, Hyundai/Boston Dynamics, Tesla) — artificialintelligence-news.com, idc.com	"Real deployments are underway"	2026-09-25 (web search)
[17]	Google Gemini API docs — robotics overview, ER 2 model page, spatial reasoning, agentic vision, Interactions API; google-gemini/robotics-samples quickstart	ER 2 capabilities, limits, data-handling rules	2026-09-18 (via `docs/gemini-robotics-model-guide.md`)
[18]	Microsoft Learn — Document Intelligence layout model; Content Understanding; multimodal search in Azure AI Search	Azure-native options	2026-09-11 (via `docs/complex-document-retrieval-visual-grounding-proposal.md`)
[19]	Model Context Protocol — modelcontextprotocol.io	MCP concepts	General reference
[21]	Yang et al., "Magma: A Foundation Model for Multimodal AI Agents," arXiv 2502.13130 (Feb 2025); `microsoft/Magma-8B` model card; `microsoft/Magma` repository; Foundry catalog entry `microsoft-magma-8b`; Microsoft Learn, "Hugging Face models in Microsoft Foundry"	Magma architecture, Set-of-Mark grounding, results, hosting via managed compute — collected in `docs/magma-readiness.md`	2026-09-29 (paper read in full)
[20]	Competitive landscape: Acuvate DiagramIQ, Pathnovo, LlamaParse, SymphonyAI IRIS Foundry, Autodesk Vault 2027 Assistant, Bluebeam Max + Claude via MCP	Chapter 11 landscape	2026-08-23 → 2026-09-25 (`RESEARCH_LOG.md`, `pitch/README.md`, web search)

Internal M-Files sources

#	Source	Used for	Verified
[9]	Confluence (Product space), "Product Strategy 2025-26: Unleashing our Knowledge Graph Like Architecture combined with AI & Microsoft to win!" — Tony Grout, Aug 2025	Knowledge-graph-like architecture; "knowledge graphs are a favored format for AI"; Microsoft alignment	2026-09-25, read directly (in a separate session)
[10]	Confluence (VMAN space), "Manufacturing Blueprint" — Ville Somppi, status 2026-09-17	Installed Base & Maintenance Knowledge (planned); non-compete stance; "records outcomes and recommends actions; it does not execute transactions"	2026-09-25, read directly (in a separate session)

#	Source	Used for	Verified
[11]	Confluence (APPS space), "2027 forward looking thoughts" — modified 2026-09-24	"Validated design work, not yet built"; Forward Deployed Engineering	2026-09-25, read directly (in a separate session)
[12]	Confluence, "CAD Integration Roadmap 2026–2029" — Sivaram Reddy, May 2026	SMB/mid-market first; "wrap around PDM, not replace"; L3 engineering knowledge graph H1 2028; drawings as "opaque blobs"	2026-09-25, read directly (in a separate session)
—	Confluence (AI space), "Beta Release for MCP server" — owner Vatsal Kariya	"Image reading is not available in Beta"	2026-08-22
—	Confluence, Aino architecture pages; "AI Foundation Platform for VTAX & VMAN: Multi-Agent Enablement"	Aino Search on text embeddings; planned Asset Management / Installed Base Intelligence Agent	2026-08-22 — **re-verify**
—	AI-Metadata team `metadata-poc` design documents, as reviewed in `docs/differentiation-plan-vs-metadata-poc.md`	Images mode ≤10 MB; no region-level attribution concept	2026-09-08
—	Sibling repository `complex-document-grounding-poc` , `docs/evaluation-plan.md`	DI adapter status; evaluation phases	2026-09-28

Known gaps in verification

- Confluence could not be re-read while this edition was written (the connector needed re-authorisation), so [9]–[12] rely on the 2026-09-25 direct read. Re-check before quoting them externally.
- The Aino and AI Foundation facts date from 2026-08-22 and may have moved.
- Microsoft Magma's hosting status was re-checked 2026-09-29. It's listed in the Foundry catalog, and Hugging Face-collection models use Foundry Managed Compute (preview) where available. Availability in our project and region, the deployment SKU and the price were not visible without signing in.
- Preview model capabilities ([17]) can change without notice.

Volume history

The book is published in numbered **volumes**. Each volume is a complete, readable snapshot of the whole book at a point in time. The Markdown chapters always show the **current** volume; every past volume's PDF is kept unchanged in `volumes/`, so you can see exactly what a reader had in front of them on a given date.

A new volume is cut when the book's substance changes: new chapters or sections, new facts that change a conclusion, or corrected claims. Typo fixes don't need a new volume.

Volume	Date	What changed	Pull requests	PDF
2 (current)	2026-09-29	Microsoft Magma. A new Chapter 4 section explains mark-based ("Set-of-Mark") grounding. Magma rows added to Chapters 1, 4, 10, 12 and 13. Glossary adds Set-of-Mark and Trace-of-Mark. New source [21]. Availability wording made precise: Foundry Managed Compute is preview, and availability depends on project, region and quota. Companion guide: `docs/magma-readiness.md`	#17, #18	volume-02-2026-09-29.pdf
1	2026-09-28	First release. 13 chapters and 2 appendices: the capability map, the concepts, the re-weighted curriculum, and the handover guide	#16	volume-01-2026-09-28.pdf

Before Volume 1 there was a six-page personal guide, *Physical Agents and World Models: Zero-to-Hero* (curated 2026-09-25). It is not in this repository.

How to cut a new volume

1. Edit the chapters on a topic branch as usual. All the steps below go in the same pull request.
2. Bump `volume` and `date` in `volume.json`.
3. Update the volume line at the top of README.md to match. The build refuses to run if they disagree.
4. Add a row at the **top** of the table above.
5. Build and freeze the PDF:

   ```
   python book/tools/build_pdf.py --release
   ```

   This writes the current PDF and also saves `volumes/volume-NN-YYYY-MM-DD.pdf`. It will not overwrite an existing frozen volume. Past volumes are never edited.